



SENCITY

도심 내 야생동물 출몰 알림 및 신고 앱

교 과 목 명 : 캡스톤디자인EDA구현프로젝트(A02)

지 도 교 수 : 최성은

제 출 일 : 2025년 12월 15일

팀 명 : F4

팀 원 : 팀장 신지윤(2302558)

팀원 박윤아(2302553)

윤누리(2302563)

- 목 차 -

1. 프로젝트 개요	01
1. 팀 구성 및 역할 분담	01
2. 개발 배경과 목적	01
3. 과제 수행 프로세스	03
4. 서비스 개요	05
2. 문제 정의 및 필요성	07
1. 문제 현황	07
2. 문제 원인 분석	07
3. 필요성	08
3. 경쟁 서비스 분석 및 차별성	10
1. SWOT 분석	10
2. 기존 서비스 비교 분석	11
3. 차별점	14
4. 프로젝트 목표 및 주요 기능	15
1. 프로젝트 목표	15
2. 사용자 모바일 애플리케이션 기능	16

3. 관리자 웹 대시보드 기능.....	25
4. 서비스 이용 흐름	36

5. 시스템 설계 및 기술 아키텍처..... 38

1. 전체 시스템 구성도	38
2. 백엔드 아키텍처	40
3. AI 모델 구조.....	41
4. 데이터베이스 설계	42
5. 데이터 흐름 구조	43
6. 영상 처리 구조	44

6. AI 모델 학습 및 프로젝트 완성도..... 45

1. 데이터 수집 및 라벨링.....	45
2. 전처리 과정.....	46
3. 모델 학습 환경 및 파라미터	50
4. 학습 결과 및 성능 평가.....	51
5. 프로젝트 완성도	54

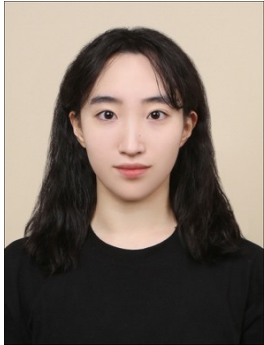
7. 기대 효과 및 사회적 가치 56

1. 기대 효과.....	56
2. 사회적 가치.....	58

8. 외부 활동 및 성과	59
1. 공모전 수상	59
2. 프로젝트 결과물	60
9. 한계점 및 향후 개선 방향	61
1. 한계점	61
2. 향후 개선 방향	62
10. 결론	63
1. 결론 및 고찰	63
11. 참고 문헌	64

1. 프로젝트 개요

1. 팀 구성 및 역할 분담

F4		
		
신지윤(PM)	박윤아	윤누리
Data Crawling Data Preprocessing Machine Learning Hardware Integration Report Writing Presentation Preparation Presentation	Application Development Back-end Development Dashboard Development API Development UI/UX Design Data Analysis Report Writing	UI/UX Design ERD Creation Application Development

2. 개발 배경과 목적

최근 급격한 기후 변화와 도시 개발로 인한 서식지 파괴로 도심 내 야생동물의 출현이 빠르게 증가하고 있다. 서울시 25개 자치구 중 24개 구에서 너구리가 관찰되었으며 최근 5년간 325마리가 구조되는 등 도심 출몰 사례가 지속적으로 보고되고 있다. 멧돼지로 인한 소방 출동 건수 또한 2022년 379건에서 2023년 649건으로 약 71% 증가하며 위험성이 확대되고 있다. 이러한 야생동물의 도심 출현은 인명 피해, 반려동물 감염, 로드킬과 같은 다양한 사회적 문제를 야기하고 있으나, 현재 관련 서비스들은 수동 신고 중심의 운영 구조와 제한적인 데이터 수집 방식으로 인해 실시간 대응과 종합적인 분석에 한계를 보이고 있다.

본 프로젝트는 이러한 한계를 해결하기 위해 AI 기반 자동 탐지 및 알림 시스템인

‘SENCITY’를 제안하며, 기존의 수동 신고 중심 체계에서 벗어나 도심 내 야생동물 출현을 능동적으로 인식하고 즉시 공유할 수 있는 지능형 대응 시스템 구축을 목표로 한다. 웹캠이 촬영한 영상 데이터는 Django 서버로 전송되며, YOLOv5s 객체 탐지 모델이 영상 속 야생동물의 위치를 실시간으로 탐지하고 출몰 상황을 구조화된 데이터로 분석한다. 분석 결과는 Flutter 앱을 통해 사용자와 지자체에 실시간으로 전달되어 현장 대응 속도를 높인다. 또한 사용자는 탐지 기록을 확인하거나 직접 사진을 업로드하여 추가 데이터를 제공할 수 있으며, EfficientNet-B1 분류 모델이 이를 분석해 종을 자동으로 판별한다. 이러한 자동 감지-분류-전송-추적 구조는 출몰 데이터를 지속적으로 확보하고 분석 정확도를 고도화하는 데이터 선순환으로 이어지며, 사고 예방과 도시 안전 확보, 생태계 보호의 균형을 동시에 달성하는 데이터 기반의 능동적 도시 안전 시스템 구축을 지향한다.

3. 과제 수행 프로세스

구분	추진내용	추진일정								
		3월	4월	5월	6월	7월	8월	9월	10월	11월
기획	서비스 선정 및 목표 수립									
분석	주요 기능 확정 및 SWOT 분석									
	경쟁사 분석 및 방향성 선정									
설계	요구분석서 & UI/UX 디자인									
	기능 명세서 / API 설계서									
개발	DataBase 설계									
	Front-end 구현									
	Back-end 구현 및 연동									
모델 학습	이미지/영상 데이터 수집									
	EfficientNet-B1 모델 학습									
	YOLOv5s 모델 학습									
	YOLOv5s 정확도 개선									
HW	아두이노 코드 작성 및 연동									
테스트	단위 테스트									
	통합 테스트									
전시회 준비	전시회 발표 ppt 작성									
	보고서 첨삭									

1. 기획 단계

도심 내 야생동물 출몰 증가 현상과 관련 통계 자료를 기반으로 문제의

심각성과 서비스 개발의 필요성을 분석하고 실시간 자동 탐지, 자동 신고, 위험 지역 시각화와 같은 핵심 목표를 설정하였다. 이와 함께 사용자 요구, 실제 활용 환경, 기술적 가능성을 종합적으로 고려하여 장기적인 데이터 축적과 활용이 가능한 방향으로 프로젝트 범위를 정의하였다.

2. 분석 단계

서비스 제공에 필요한 핵심 기능을 도출하고 우선순위를 정리하였으며, 데이터 수집과 활용 관점에서 프로젝트의 강점과 한계를 파악하기 위해 SWOT 분석을 수행하였다. 또한 유해동물시민세이프존, 굿로드, 포스코DX AI 영상 인식 기술 등 기존 서비스 사례를 비교 분석하여, 수동 신고 중심 구조와 제한적인 데이터 활용이라는 한계를 확인하고 이를 보완할 수 있는 SENCITY의 차별적 방향성을 설정하였다.

3. 설계 단계

서비스 기능 요구사항과 데이터 흐름을 정리한 문서를 작성하고, 사용자 흐름을 기반으로 한 UI/UX 와이어프레임과 디자인 시안을 제작하였다. Django 기반 REST API를 중심으로 탐지 데이터 생성, 데이터베이스 저장, 시각화, 알림 전송으로 이어지는 전체 데이터 파이프라인 구조를 설계하였으며, 서버, 모바일 앱, AI 모델 간 연동 방식을 기능 명세서와 구조도로 문서화하여 향후 분석 및 예측 기능 확장이 가능하도록 하였다.

4. 개발 단계

탐지 결과, 사용자 정보, 신고 기록을 저장하기 위한 SQLite 기반 데이터베이스를 구축하고, Flutter 기반 모바일 앱에서 회원가입과 로그인, 지도, 알림, 신고, 기록 조회 등 핵심 기능을 구현하였다. Django 백엔드에서는 AI 분석 결과 저장과 데이터 관리 기능을 구현하고, 관리자 대시보드를 통해 출몰 현황과 통계 정보를 확인할 수 있도록 하였다. 이를 통해 프론트엔드, 백엔드, AI 모델이 실시간으로 데이터를 주고받는 통합 연동 구조를 완성하였다.

5. AI 모델 학습 단계

Kaggle 오픈소스 데이터, 구글 크롤링을 통해 이미지와 영상 데이터를 수집하고 전처리를 수행한 뒤, EfficientNet-B1 기반 이미지 분류 모델과 YOLOv5s 기반 객체 탐지 모델을 학습하였다. 오탐과 미탐 문제를 줄이기 위해 데이터 증강, 하이퍼파라미터 튜닝, Confidence 및 NMS 값 조정을 반복적으로 수행하였으며, 야간과 악천후 환경에서도 안정적인 인식이 가능하도록 다양한 환경 조건을 반영한 재학습을 진행하였다.

6. 하드웨어 구현 단계

RTSP 기반 IP 웹캠을 활용하여 실시간 영상 스트리밍을 서버와 연동하였고, 전송 지연과 프레임 유실과 같은 실환경 문제를 테스트와 튜닝을 통해 개선하여 실제 현장에서 수집되는 데이터의 안정성과 품질을 확보하였다.

7. 테스트 단계

기능 단위 테스트를 통해 각 모듈의 안정성을 검증하고 앱, 서버, AI 모델, 하드웨어를 포함한 전체 시스템 통합 테스트를 수행하였다. 또한 야간, 악천후, 동물 이동 속도 변화 등 다양한 조건에서 재현성 테스트를 진행하며 오탐과 미탐 비율, 알림 지연 시간, 서버 부하와 같은 실제 서비스 지표를 기준으로 품질을 검증하였다.

8. 전시회 및 공모전 준비 단계

프로젝트 소개용 PPT와 포스터를 제작하고 시연 영상 촬영과 테스트 결과 정리를 통해 프로젝트의 완성도를 높였다. 또한 보고서를 작성하여 지도 교수의 피드백을 반영하고, 발표 대본과 질의응답 구조를 준비함으로써 전시 및 공모전 발표에 적합한 형태로 결과물을 정리하였다.

4. 서비스 개요

SENCITY는 사용자 모바일 앱, 백엔드 서버, AI 분석 모듈, 관리자 대시보드, 푸시 알림으로 이어지는 단일 파이프라인 구조로 동작하는 통합 서비스이다. 사용자는 모바일 앱을 통해 사진 기반 신고를 수행할 수 있으며, 업로드된 이미지는 AI 모델에 의해 자동으로 분류되고 GPS 정보를 기반으로 위치가 매핑된다. 동시에 웹캠을 통해 수집된 스트리밍 영상은 AI 분석 모듈에서 실시간으로 처리되어 별도의 사용자 제보 없이도 출몰 데이터를 생성한다. 이러한 데이터는 Django REST API 기반의 백엔드 서버에서 통합 관리되며, 관리자 대시보드를 통해 신고 현황 확인, CCTV 탐지 결과 모니터링, 공지 및 위험 알림 발송에 활용된다. 고위험 종이 탐지될 경우에는 FCM을 통해 인근 사용자에게 실시간 경고가 전달되어 즉각적인 대응을 가능하게 한다.

이와 같은 시스템 구조를 바탕으로 SENCITY의 핵심 아이디어는 도심 내 야생동물 출몰 데이터를 자동으로 수집하고 분석하며 전달하는 지능형 데이터 플랫폼을 구축하는 데 있다. 기존 신고 시스템이 시민의 수동 제보에 의존하여 데이터 누락과 대응 지연이 발생하는 것과 달리, SENCITY는 AI 기반 자동 탐지 모델인 EfficientNet-B1과 YOLOv5s,

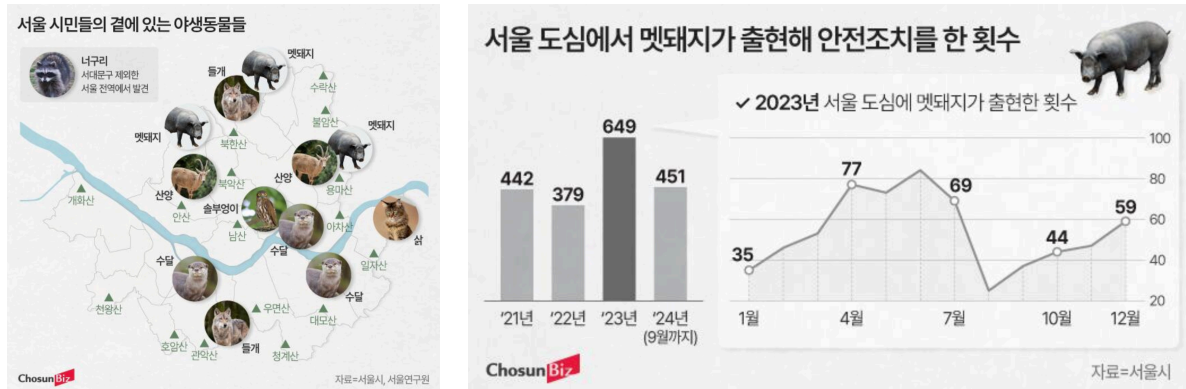
그리고 웹캠, 서버, 모바일 앱을 유기적으로 연결한 통합 데이터 파이프라인을 통해 실시간 출몰 정보를 자동으로 확보하도록 설계되었다. 이를 통해 출몰 데이터가 지속적으로 축적되며, 향후 정책 및 행정 활용이 가능한 생태 데이터 기반을 마련하는 것을 핵심 목표로 한다.

SENCITY의 주요 특징은 다음과 같은 구조적 차별성에서 나타난다. 먼저 웹캠 스트리밍 영상을 YOLOv5s 모델이 실시간으로 분석하여 동물의 위치와 종류를 자동으로 판별함으로써 사용자의 제보 없이도 출몰 데이터를 생성하는 능동형 감지 시스템을 구현하였다. 탐지된 데이터는 Django 서버를 거쳐 즉시 데이터베이스에 저장되고, 모바일 앱을 통해 사용자에게 실시간으로 전달되며 지도 기반 위험 지역 시각화로 제공된다. 이는 단순한 알림 전달을 넘어 데이터 기반 상황 인지와 의사결정을 가능하게 한다. 또한 EfficientNet-B1 모델을 활용한 사용자 신고 이미지 분류 기능을 통해 사용자 참여 기반의 데이터 확장성을 확보하였으며, 이는 AI 학습 데이터의 품질과 다양성을 높여 모델 성능 개선으로 이어진다. 더 나아가 공공데이터, AI 분석 데이터, 사용자 데이터를 결합한 종합 분석이 가능하도록 설계하여 월별 출몰 추세 분석이나 위험도 예측과 같은 데이터 활용 중심의 관리자 대시보드를 제공함으로써, 서비스가 도시 생태계 분석 도구이자 정책 지원 플랫폼으로 확장될 수 있는 기반을 마련하였다.

결과적으로 SENCITY 는 실시간 대응 속도 향상과 오인 신고 감소를 동시에 달성하고, 축적된 데이터를 기반으로 한 정책적 의사결정 지원까지 가능하게 하는 AI, IoT, 데이터 분석이 결합된 통합 시스템으로서 기존 야생동물 신고 서비스의 한계를 보완하고 도심 안전을 위한 데이터 중심 접근을 제시하는 실효성과 창의성을 지닌 플랫폼이다.

2. 문제 정의 및 필요성

1. 문제 현황



(단위: 건)

구분	고라니	너구리	멧돼지	노루	오소리	개	고양이	기타
2019년	11,638	1,731	315	557	141	1,016	4,661	1,338
2020년	6,864	1,415	134	610	136	654	3,008	2,286
2021년	10,847	2,291	124	872	245	1,605	17,527	3,750
2022년	16,075	3,353	136	967	376	2,257	31,880	8,945
2023년	18,267	4,011	192	968	510	2,575	38,143	14,612
합계	63,691	12,801	901	3,974	1,408	8,107	95,219	30,931

[정책인사이트] 서울누비는야생동물...암사동애삼,연대뒷산애산양,남산애솔부엉이-조선일보손덕호,홍다영기자

2019 ~ 2023년 사이 주요 야생동물 출몰 건수는 지속적으로 증가하였다. 특히 고라니는 2021년 대비 2023년에 약 68% 증가, 너구리는 75% 증가, 멧돼지는 26% 증가하는 등 도심 출몰이 빠르게 확대되고 있다. 이는 단순 출몰뿐 아니라 도로 로드킬, 2차 사고로 이어질 위험까지 증가시키며 시민 안전에 대한 부담을 높이고 있다.

2. 문제 원인 분석

도심 야생동물 출몰 문제가 심화되는 주요 원인은 다음과 같다.

1) 기존 신고 체계의 비효율성

- 전화/문자 중심 신고로 실시간성이 낮고 정확한 대응이 어려움
- GPS 위치가 자동으로 기록되지 않아 부정확한 위치 정보 전달

2) 데이터 기반 관리 부재

- 부처/기관별로 출몰 데이터가 분산되어 있어 통합 관리 어려움
- 장기적 정책 분석/예측 모델 개발이 불가능한 구조

3) 자동 감지 기술 미도입

- CCTV가 단순 관제 용도로만 활용되어 출몰 여부 자동 탐지가 이루어지지 않음
- 로드킬, 야간 출몰 등 긴급 상황을 실시간 탐지할 수 없음

4) 대응 체계의 속도 저하

- 데이터 수집-판단-전달 과정이 수작업 기반이라 즉시 경고가 불가
- 시민, 지자체, 관제센터 간 정보 전달 속도 지연

결과적으로, 증가하는 출몰 위험에 비해 신고-판단-전달 과정이 느리고 단절된 구조가 문제로 드러난다.

3. 필요성

이러한 문제를 해결하기 위해서는 기존 신고 방식의 한계를 보완할 수 있는 AI 기반 자동 분석, GPS 기반 위치 매핑, CCTV 영상 자동 탐지, 실시간 푸시 알림이 결합된 통합 플랫폼이 필요하다.

- AI 기반 자동 분석: 종 판별 정확도 향상, 오인 신고 감소
- GPS 자동 위치 인식: 신고 즉시 정확한 위치 확보
- CCTV 자동 탐지 연동: 사람 개입 없이 출몰 즉각 감지
- FCM 실시간 경고 시스템: 위험 상황 즉시 시민에게 전달

SENCITY는 이와 같은 요구를 충족하여

1. 실시간 감지 및 조기 경보: AI 기반 자동 감지로 야생동물 출몰을 실시간으로 파악하고, 시민에게 즉각적인 알림을 제공하여 사전 대응 가능
2. 시민 안전 확보: 위치 기반 푸시 알림으로 출몰 지역 인근 시민들에게 선제적 정보를 제공하여 안전사고 예방
3. 효율적인 관리 체계 구축: 관리자 대시보드를 통해 출몰 현황을 모니터링하고, 데이터 기반 대응 전략 수립 가능
4. 데이터 추적 및 분석: 출몰 패턴, 시간대, 지역별 통계 데이터를 추적하여 장기적인 야생동물 관리 정책 수립에 활용
5. 사회적 인식 제고: 시민들에게 야생동물 출몰 현황과 대응 방법을 지속적으로 안내하여 환경 보호 및 공존 의식 향상

이와 같은 사회적·행정적 가치를 함께 제공한다.

결과적으로 SENCITY는 도심 내 야생동물 출현을 단순히 감지하고 전달하는 수준을 넘어, AI 기반 자동 분석과 위치 정보, 실시간 알림을 결합한 통합 대응 체계를 통해 시민의 안전을 선제적으로 확보하고 행정 기관의 대응 효율을 높이며, 장기적인 데이터 축적을 통해 지속 가능한 야생동물 관리와 정책 수립을 가능하게 하는 핵심 플랫폼으로 기능한다.

3. 경쟁 서비스 분석 및 차별성

1. SWOT 분석

• AI 기술 발전 : 영상인식/딥러닝 정확도 향상 → 실시간 사진 분석 가능 → 즉각적인 출현 알림 제공 • 위치정보 기술 보편화 : GPS/지도 연동이 정교해짐 → 로드 킬 다발 지점/위험 지역 시각화 가능 • 사회적 필요성 반영 : 야생동물 사고 급증으로 기술 활용 필요성이 커짐	• 데이터 구축 필요성 : AI 활용을 위해선 대규모 사진/영상 데이터 라벨링 필수 → 초기 비용·노력이 요구됨 • 초기 사용자 참여 미흡 : 참여가 저조하면 데이터 수집이 지연 → 알림·예방 효과가 제한됨 • 기술적 제약 : 야생동물 분류 정확도를 높이려면 장시간 학습/성능 자원이 필요 → 단기간 내 상용화 어려움
• 지자체/환경부 협력 가능 공공기관과 데이터 공유 → 정책·관리 지원 가능 • AI 기술의 발전 YOLO 등 모델 적용 시 다양한 동물 분류 및 추적 가능 • 확장성 산불, 홍수 등 다른 위험 요소 알림 시스템으로 발전 가능	• 데이터 정확성 문제 잘못된 신고/알림으로 불필요한 경고가 발생할 수 있음 • 야생동물 행동 예측 불확실성 야생동물의 행동은 예측 어려움 → 알림 효율성 저하 가능성

1. 강점(Strength)

- 이미지 및 영상 기반 AI 자동 분석(EfficientNet-B1, YOLOv5s) 적용
- GPS 기반 정확한 위치 정보 확보
- 앱-서버-AI-대시보드-FCM의 통합 파이프라인 구축
- 도심 야생동물 사고 증가로 사회적 필요성 확대

2. 약점(Weakness)

- 학습 데이터 구축과 모델 운영에 필요한 초기 비용
- 환경/야간/저해상도 상황에서 발생할 수 있는 탐지 편차
- 지속적인 모델 업데이트와 운영 관리 필요

3. 기회(Opportunity)

- 지자체/산림청/환경부 등 공공기관 협력 가능성
- 스마트시티, 재난 대응 플랫폼으로 확장 가능성
- 지자체/공공기관에서의 AI 및 CCTV 인프라 확대 추세

4. 위협(Threat)

- 데이터 품질 저하 시 발생 가능한 오경보 위험
- 야생동물의 불규칙적 행동으로 인한 예측 어려움
- 유사 모바일 신고 플랫폼 등장 가능성

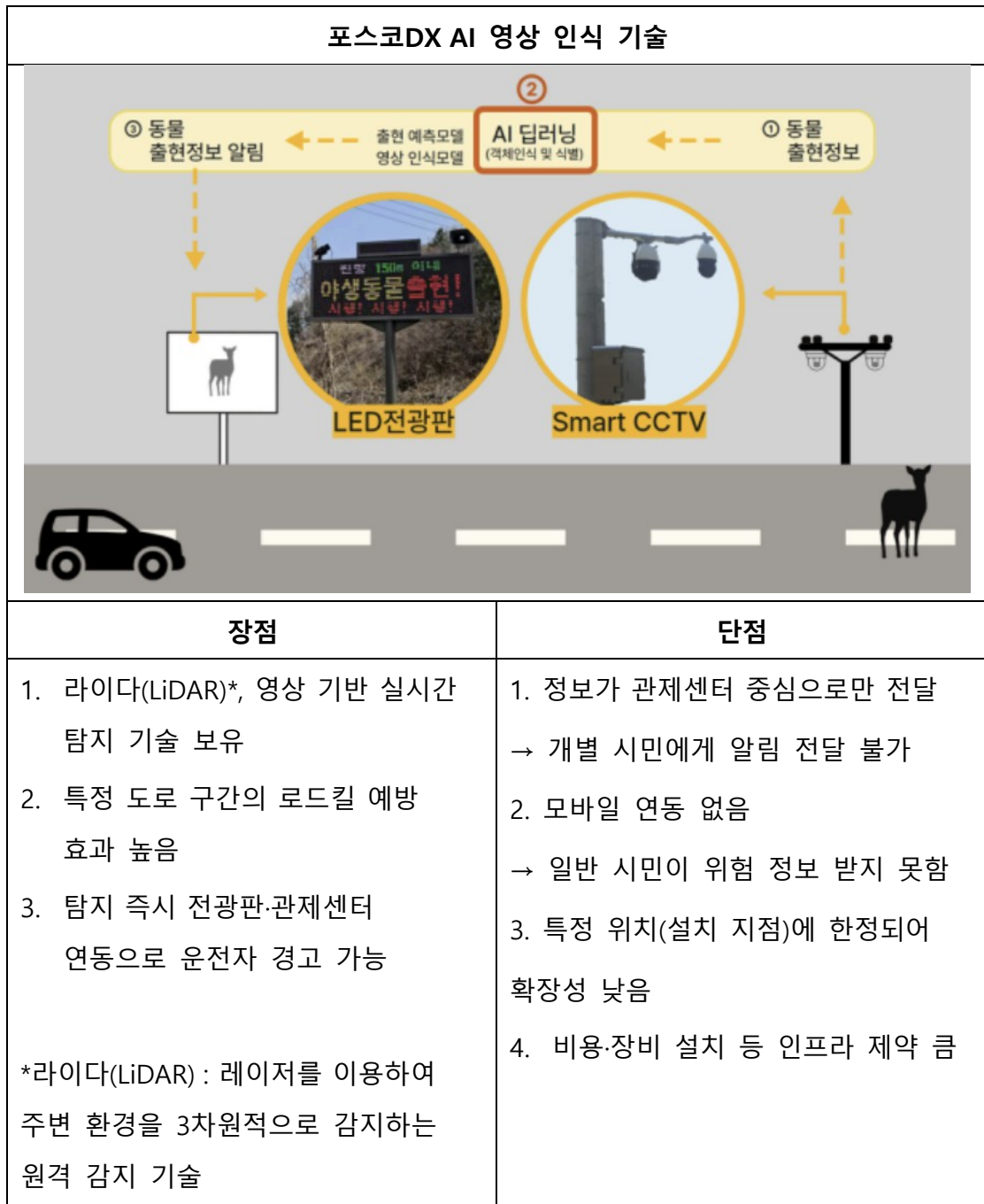
2. 기존 서비스 비교 분석

1) 모바일 기반 신고 서비스

유해동물 시민 세이프존	
	장점
	1. 직관적 UI/UX로 사용성 높음 2. 비콘 기술 기반 지역 알림 기능 제공 3. 기본적인 출몰 지도 및 신고 기능 구성
	단점
	1. 멧돼지 중심 서비스 → 종 다양성 대응 불가 2. AI 자동 분석 기능 부재 3. CCTV 연동/실시간 분석 등 고도화 기능 없음 4. 사용자 참여 감소 시 데이터 축적 어려움

굿로드	
	장점
	1. 위치 정보 + 사진 기반 로드킬 신고 기능 2. 녹색연합·국립생태원 등 전문기관 협력 3. 오프라인 환경에서도 동작 가능한 추적 기능 제공
	단점
	1. 로드킬 중심 서비스 → 도심 야생동물 출몰 대응 한계 2. 전문 조사자 승인 필요 → 일반 시민 사용성이 떨어짐 3. AI 자동 분석 기능 부재 4. 생활권 위험 관리 기능 없음

2) CCTV 영상 분석 기반 기술(포스코DX)



3. 차별점

기존 야생동물 신고 시스템은 시민의 수동 제보에 의존하는 구조로 인해 신고 지연이나 데이터 누락 문제가 빈번하게 발생해 왔다. 이에 비해 본 프로젝트 SENCITY는 YOLOv5s 기반 실시간 자동 탐지, EfficientNet-B1 이미지 분류 모델, GPS 기반 위험 지도, FCM 알림 시스템, 웹캠 스트리밍 기술을 결합하여 야생동물 출몰 정보를 자동으로 생성하고 저장하며 분석과 활용까지 이어지는 완전 자동화된 생태 안전 시스템을 구축하였다. 이를 통해 사람의 개입 없이도 출몰 상황을 즉각적으로 인지하고 공유할 수 있으며, 데이터 수집과 관리 과정 전반의 신뢰성과 효율성을 크게 향상시켰다.

또한 SENCITY는 특정 지역이나 일부 동물에 한정된 기존 서비스와 달리 10종 이상의 야생동물을 자동으로 분류할 수 있고, 악천후 환경에서도 대응 가능한 모델과 실시간 신고 자동화, 모바일 즉시 알림 기능을 통합하여 범용성과 실효성을 동시에 확보하였다. 이러한 통합적 설계는 단순 알림 서비스 수준을 넘어 출몰 패턴 분석과 위험도 추정, 데이터 기반 대응 전략 수립까지 가능하게 하며, 도심 야생동물 관리에 있어 차별화된 데이터 중심 플랫폼으로 기능한다.

- 1) AI + GIS + CCTV를 통합한 사용자 참여형 시스템
 - 이미지 기반 종 자동 판별(EfficientNet-B1)
 - CCTV 실시간 탐지(YOLOv5s)
 - GPS 자동 위치 매핑 → 지도 기반 시각화
- 2) 고위험 출몰 시 시민에게 즉시 FCM 알림 전달
 - 관제 센터 중심 전달 방식의 한계를 해결
 - 시민 개개인의 실시간 대응 능력 향상
 - 모바일 중심의 생활권 안전 시스템 구
- 3) 데이터 기반 정책 분석이 가능한 구조
 - 신고 데이터 + CCTV 탐지 데이터를 단일 DB에서 관리
 - 지자체/기관이 활용할 수 있는 실질적 정책 가치 제공
 - 종별·지역별·시간대별 패턴 분석 가능
- 4) 단일 파이프라인 기반의 운영 효율성
 - 앱 → 서버 → AI → DB → 대시보드 → FCM
 - 중간 단계 누락 없이 자동화된 처리
 - 확장성 높은 구조(다른 위험 감지 기능 추가 가능)

4. 프로젝트 목표 및 주요 기능

1. 프로젝트 목표

본 프로젝트의 목표는 기존 야생동물 신고 시스템의 구조적 한계를 보완하여, 사용자가 현장에서 즉시 판단하고 신고하며 대응할 수 있는 실질적인 안전 정보 제공 체계를 구축하는 데 있다. 이를 위해 현재 위치 기반의 출몰 위험도와 위험 지역 정보를 지도 형태로 제공하여 사용자가 상황을 빠르게 인지할 수 있도록 하고, 긴급 상황에서도 최소한의 조작만으로 즉시 신고가 가능한 간단한 인터페이스를 구현하고자 한다. 또한 시민이 남긴 신고 데이터가 실시간으로 위험도 지도에 반영되고 다시 개인 맞춤형 안전 정보로 제공되는 순환 구조를 통해, 시민과 관리 기관 간 데이터가 상호 활용되는 체계를 구축한다. 이와 함께 목적, 촬영, 자동 분류, 자동 위치 입력, 전송으로 이어지는 단일 흐름의 신고 프로세스를 설계함으로써 판단과 입력 과정을 최소화하고, 신고 지연이나 위치 오류와 같은 문제를 줄이는 것을 목표로 한다. 이를 통해 신고부터 대응까지 전 과정이 현장 중심으로 단순화된, 실효성 높은 도심 야생동물 대응 시스템을 구현하고자 한다.

1) 즉각적인 위험 판단 정보 제공

- 현재 위치의 위험도와 출몰 위험 지역을 지도 기반으로 확인
- 긴급 상황에서 바로 신고할 수 있는 간단한 인터페이스 제공

2) 시민·관리기관 데이터의 상호 활용 구조 구축

- 시민이 남긴 신고가 즉시 위험도 지도에 반영 → 개인 맞춤 안전 정보로 제공
- 데이터가 단방향이 아닌 순환 구조로 작동하여 활용성과 정확도 향상

3) 신고부터 대응까지의 현장 중심 단순화

- 목적 → 촬영 → 자동 분류 → 자동 위치 입력 → 발송의 단일 흐름
- 판단, 입력 과정 최소화로 신고 누락 및 위치 오류 감소

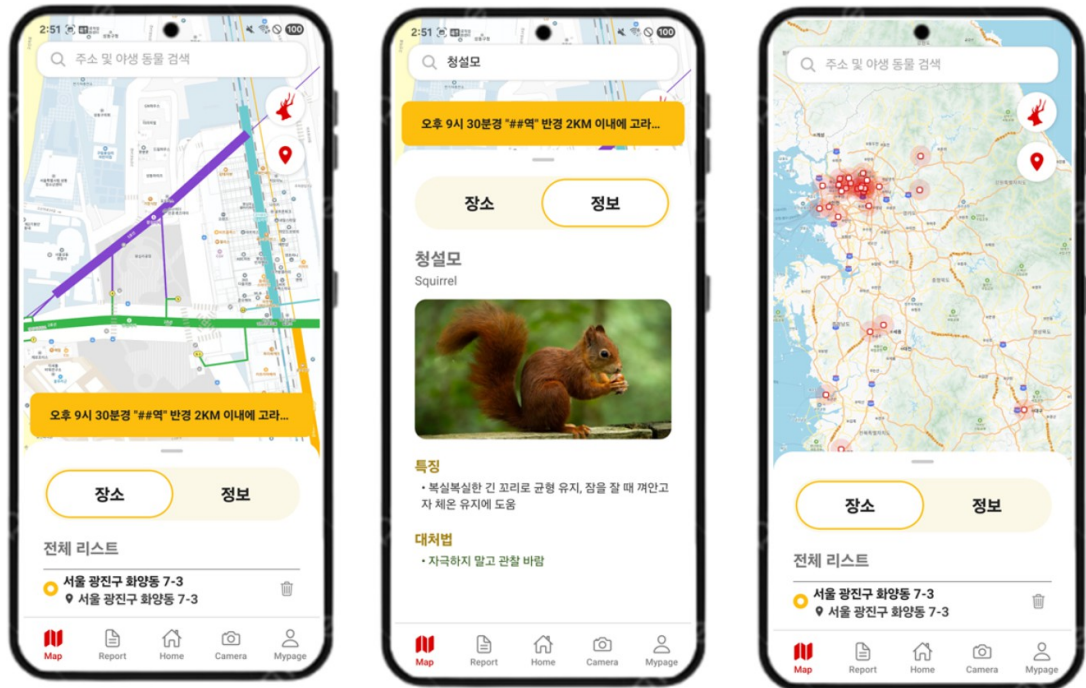
2. 사용자 모바일 애플리케이션 기능

사용자 모바일 애플리케이션과 관리자 웹 대시보드로 구성되며, Django REST API를 통해 실시간으로 연동된다. 주요 기능은 다음과 같다.

- 사용자 모바일 화면

홈 화면	
	사용자가 앱을 실행했을 때 현재 위험 상황과 주요 기능을 한눈에 파악할 수 있도록 구성한 핵심 화면이다.
	주요 요소 - 핵심 통계 카드(KPI 카드) 사용자 주변의 출몰 현황을 요약해 보여주며, 안전 결정에 직접적으로 도움이 되는 정보를 제공한다. - 뉴스 카드 최근 야생동물 관련 보도를 사용자에게 전해 상황 이해도를 높인다. - 지도 카드 카카오맵 지도 기반이며 클릭 시 지도 화면으로 넘어갈 수 있다.

지도 화면



지도 화면은 사용자가 현재 위치, 주변 출몰 정보, 특정 지역 검색, 위험 알림을 한 번에 확인할 수 있도록 설계된 핵심 기능 화면이다. 카카오맵 API를 기반으로 직관적인 지도 탐색이 가능하며, 사용자의 상황 판단을 돕는 정보를 시각적으로 제공한다.

주요 기능

- 출몰 위치 표시: 최근 신고된 동물 위치를 지도에 마커 형태로 시각화하여 주변 위험도를 바로 확인할 수 있다.
- 현재 위치 확인: 카카오맵 기반 GPS 연동을 통해 사용자의 현재 위치를 즉시 표시하여 이동 중에도 위험 구간을 피할 수 있다.
- 검색 기능: 사용자가 직접 주소나 지역명을 입력하여 장소를 저장할 수 있고, 야생동물 출몰 정보를 조회할 수 있다.
- 실시간 알림 연동: 위험 동물이 탐지되면 관리자 대시보드에서 발송한 알림을 실시간으로 표시하여 즉각적인 대응이 가능하다.

설계 의도

사용자 주변의 위험 정보를 지도 중심으로 직관적으로 전달해 안전성을 강화한다.

신고 통계 및 기록 조회 화면



신고 통계 및 기록 조회 화면은 사용자가 그동안의 신고 데이터를 기반으로 출몰 패턴, 개인 신고 이력을 한눈에 확인할 수 있도록 구성된 분석 화면이다.

주요 기능

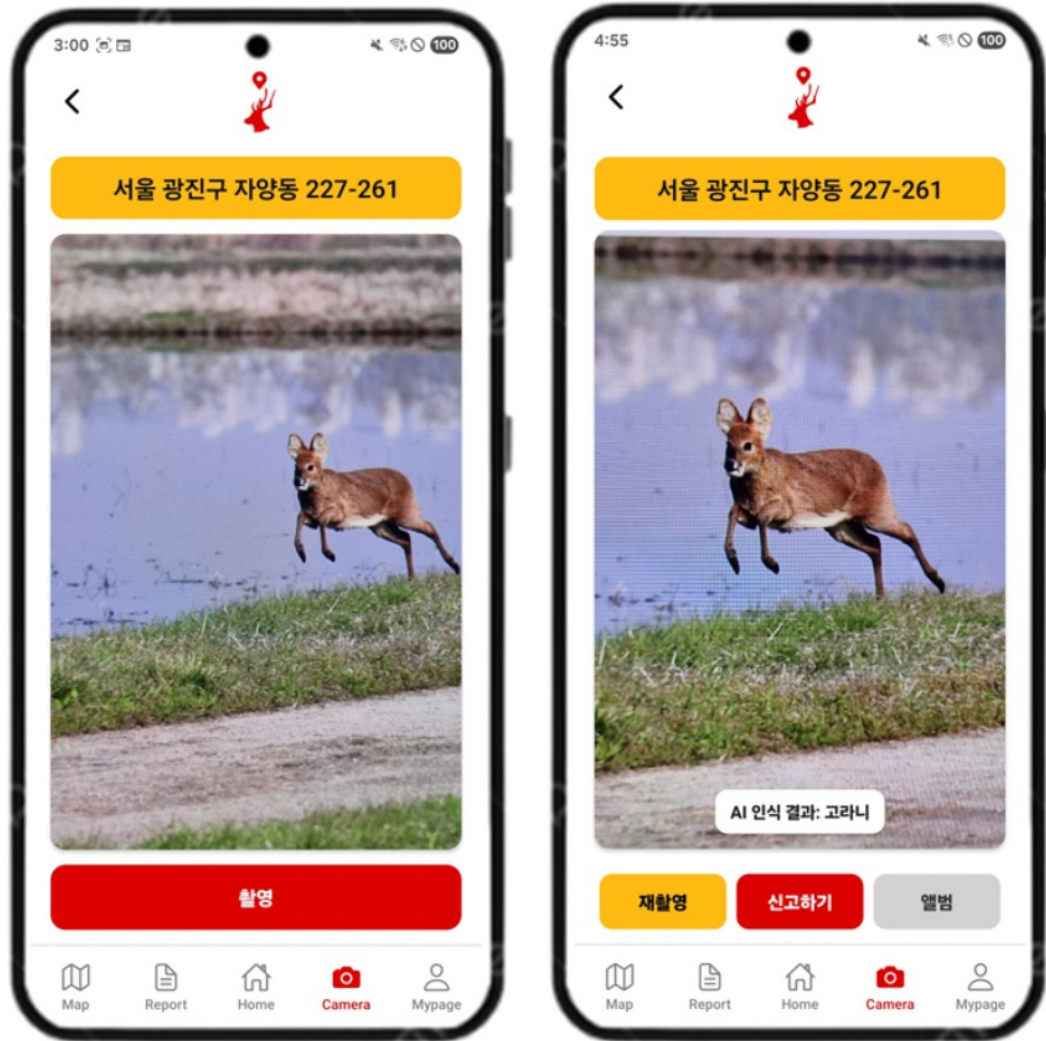
- 종별·시간대별 통계 제공: 신고된 동물 비율을 도넛차트, 막대 그래프로 시각화하여 지역 위험도를 직관적으로 확인할 수 있다.
- 개인 신고 기록 조회: 사용자가 직접 남긴 신고를 시간순으로 정렬하여 이미지, 위치, 날짜 등 상세 정보를 한 번에 확인할 수 있다.
- 위험 패턴 분석: 최근 증가한 동물 종류나 특정 시간대의 출몰 증가 등 변화된 패턴을 바로 확인하여 사용자 행동 안전성 향상에 도움을 준다.

- 관리기관 처리 현황 확인: 사용자가 올린 신고가 '확인 중 / 보류 / 처리 완료' 상태인지 즉시 확인할 수 있어 신고 이후의 진행 상황을 투명하게 파악할 수 있다.

설계 의도

단순 기록 나열이 아닌 의사결정에도 도움되며, 사용자 맞춤형 안전 관리가 가능하도록 설계하였다.

신고 화면

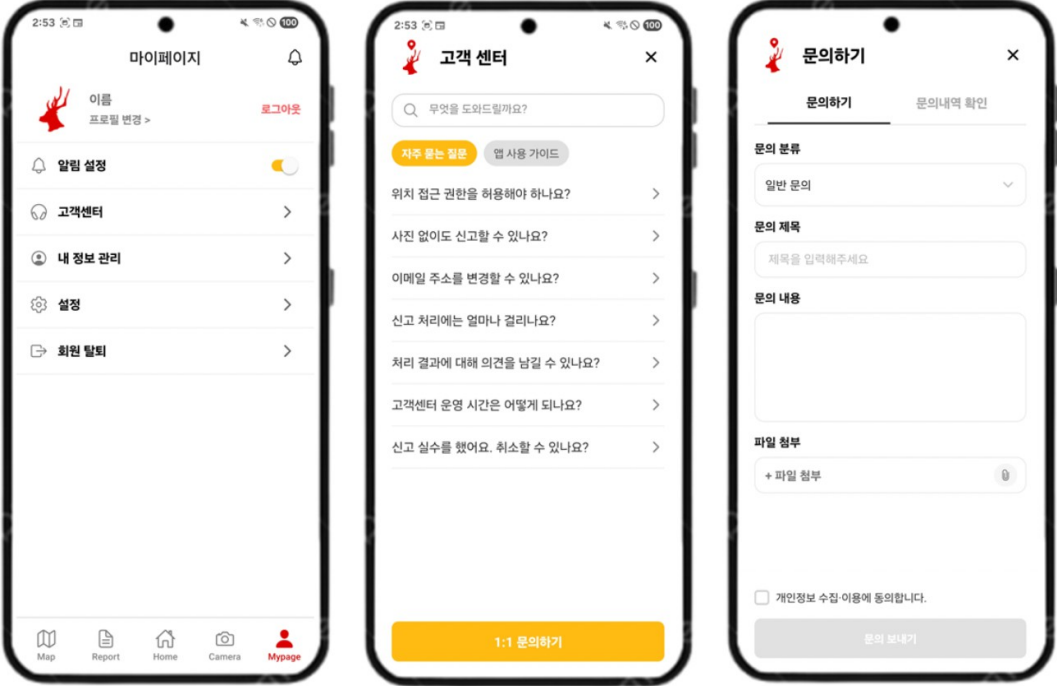


신고 화면은 사용자가 현장에서 즉시 촬영하고 빠르게 신고할 수 있도록 구성된 핵심 기능이다. 사진 촬영 직후 AI가 자동으로 동물을 분류하고 위치 정보까지 자동 입력되어, 최소한의 조작만으로 신고가 가능하다.

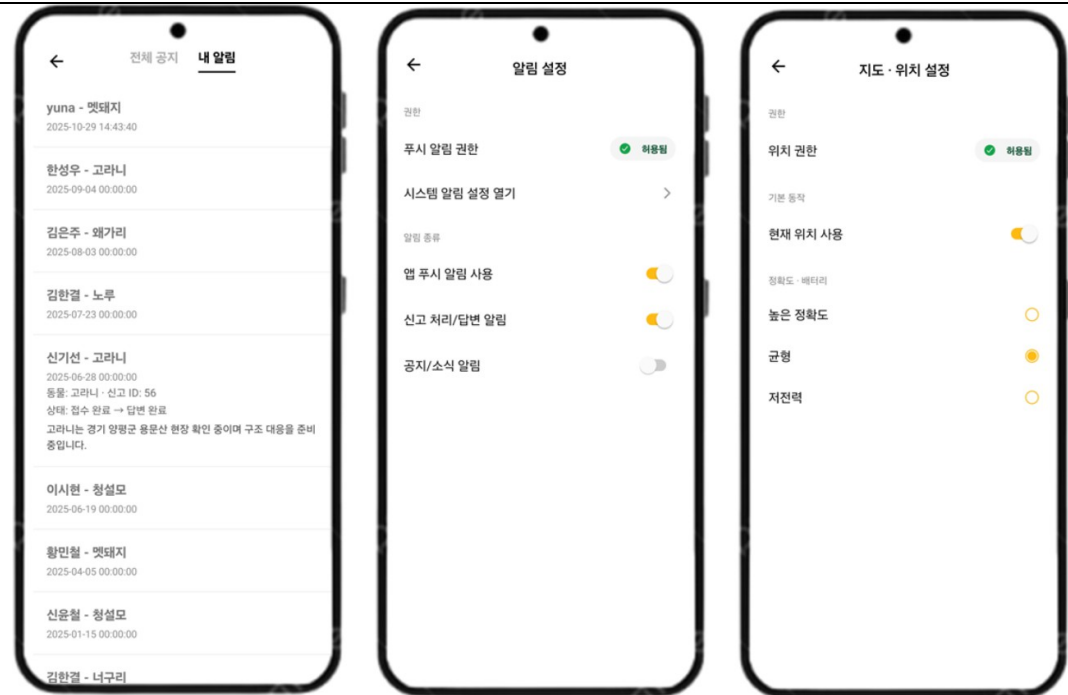
주요 기능

- 실시간 촬영 및 AI 자동 분류: 촬영된 이미지에 대해 AI가 즉시 종을 예측하여 사용자가 별도 선택 없이 신고를 진행할 수 있다.
- GPS 기반 위치 자동 입력: 촬영 시점의 위치가 자동 입력되므로 손으로 입력할 필요가 없다.
- 오류 방지 기능: AI 결과가 낮은 신뢰도일 경우 자동 신고가 중지되고, 잘못된 데이터는 학습 데이터로 재학습의 바탕이 된다.

설계 의도
'찍고 → 확인하고 → 제출'의 단순한 흐름으로 현장 대응 속도 향상하며 누구나 쉽게 신고할 수 있도록 구성하였다.

마이페이지 화면	
	
마이페이지 화면은 사용자가 개인 정보를 관리하고 앱 이용 중 필요한 설정·문의·계정 정보를 한곳에서 확인할 수 있도록 구성된 관리 중심 화면이다. 신고 기능 외에도 서비스 이용 지속성을 높이기 위한 필수 기능들을 모아두었다.	
주요 기능	
- 알림 설정: 위치 기반 위험 알림, 공지 알림 등 알림 여부를 선택할 수 있는 사용자 맞춤 설정을 제공한다. - 고객센터 및 문의 기능: 이용 중 불편 사항이나 개선 요청을 1:1 문의로 전달할 수 있으며, 답변 내역을 앱에서 바로 확인 가능하다. - 개인정보 및 계정 관리: 이름, 연락처, 이메일 등 사용자 계정을 안전하게 관리할 수 있다. - 설정: 위치·알림·오프라인 모드 등 개인별 설정을 제공한다.	
설계 의도	
사용자 계정 정보·문의·설정 기능을 분리하지 않고 한 화면에 통합하여 접근성을 높이며 지속적 서비스 이용을 위해 기본 관리 기능을 체계적으로 제공한다.	

마이페이지 상세 화면



마이페이지 상세 화면은 사용자가 앱 이용을 위해 필요한 공지 확인, 알림 설정, 위치/지도 설정, 계정 관리 기능을 세분화해 제공하는 관리 중심 영역이다. 사용자가 원하는 기능을 명확하게 찾을 수 있도록 화면을 항목별로 분리하여 구성하였다.

주요 기능

- 공지 화면: 전체 시스템 공지와 개인 신고 관련 알림을 구분해 제공하며, 사용자가 필요한 정보를 빠르게 확인할 수 있다.
- 알림 설정 화면: 푸시 알림 여부, 위치 기반 알림, 신고 처리 알림 등 사용자가 받고 싶은 알림만 선택적으로 설정할 수 있어 불필요한 알림을 최소화한다.
- 지도·위치 설정 화면: GPS 사용 여부, 지도 표시 옵션 등을 조절할 수 있으며, 사용자가 선호하는 방식으로 위치·지도 기능을 맞춤 설정할 수 있다.
- 회원 탈퇴 기능: 개인정보 보호를 위해 탈퇴 전 확인 단계를 거치며, 모든 개인 데이터가 안전하게 삭제되도록 설계하였다.

설계 의도

설계 의도: 각 기능을 항목별로 분리하여 사용자가 필요한 설정에 빠르게 접근할 수 있도록 개인화된 서비스를 제공하며, 앱 이용 과정에서 발생하는 불편을 최소화하고, 사용자 경험을 안정적으로 유지한다.

3. 관리자 웹 대시보드 기능

홈 화면

관리자 페이지

SENACITY

신고 처리 관리자

신고 관리

CCTV 관리

신고 처리 및 관리

설정

사용자 관리

문자 알림

데이터 분석 및 통계

관리자 센터

공지 사항

대시보드

현황을 일눈에 확인하고 바로 조치하세요

총 신고

73

현재 보기

오늘 접수

0

오늘 날짜 기준

처리율

40

checking 중인 신고

처리율

45.2 %

한눈에 보기

실시간 상황



CCTV 원격으로 이동

최근 이벤트가 없습니다.

최근 신고

ID	종류	지역	상태	접수시간	사용자
97	땃대지	서울특별시 성동구 불광로 200	checking	2025-11-12 11:02	yuna
95	땃대지	서울특별시 관악구 아차산로 272	completed	2025-10-23 16:17	yuna
94	배가리	서울 관악구 자양동 794-10	completed	2025-10-15 02:21	yuna
70	고라니	한강변	checked	2025-09-01 00:00	한성우
69	목재비	수변공원	checking	2025-08-24 00:00	김병찬
68	고라니	남양 토지공원	completed	2025-08-23 00:00	문지현

최근 공지

현재 공지

상대 변경 안내

2025-10-20T14:03:00Z(2025-10-20) - 제1호 - 495

신고 완료되었습니다. 안전한 곳으로 대피하세요

서비스 점검 안내

2025-10-20T11:00:00Z(2025-10-20) - 제2호 - 478

오늘 22:00~23:00 점검 예정입니다.

상대 변경 안내

2025-10-20T10:00:00Z(2025-10-20) - 제3호 - 479

고라니는 서울 강서구 한강변 한강 공원 용이로 구조 대응을 준비 중입니다.

신고한 사용자 TOP

사용자	신고 건수
김병찬	4
신재호	3
yuna	3
한성우	2
문지현	2

관리자 웹 대시보드는 현장에서 접수된 신고를 실시간으로 확인하고, 출몰 패턴과 관리자 조치를 한눈에 파악할 수 있도록 구성된 통합 운영 화면이다. 신고 관리, CCTV 실시간 조회, 통계 분석 등 주요 기능을 직관적으로 배치해 관리자 업무 효율을 극대화하도록 설계하였다.

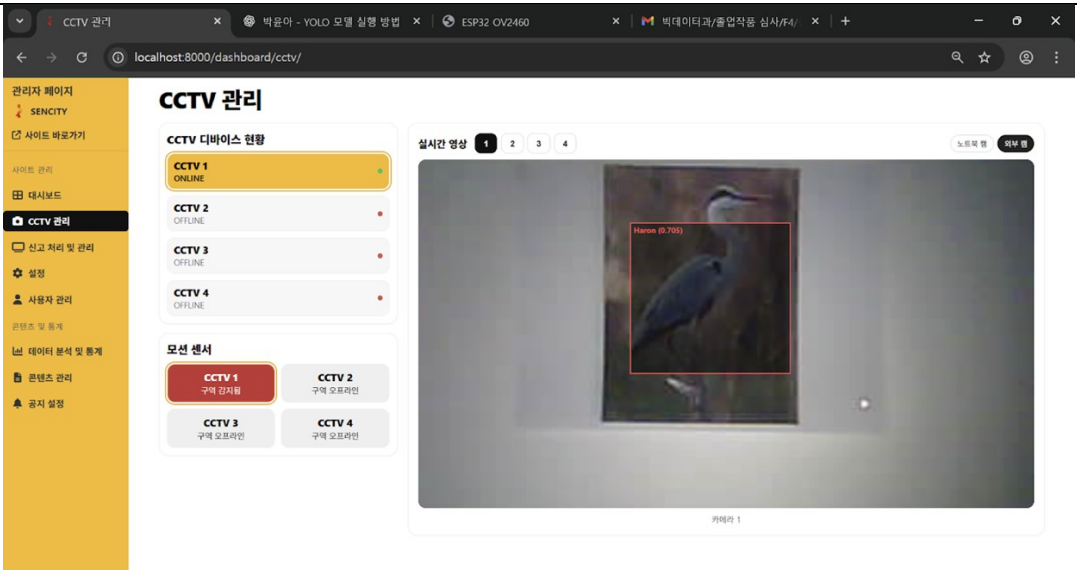
주요 기능

- 실시간 신고 현황 표시: 새로 접수된 신고 건을 시간순으로 표시하며, 신고 위치/종류/이미지를 바로 확인할 수 있어 즉각적인 대응이 가능하다.
- KPI / 핵심 지표 대시보드: 전체 신고 건수, 처리율 등 운영에 필요한 핵심 지표를 요약해 제공하여 관리자가 빠르게 상황을 파악할 수 있다.
- 신고 상세 조회 및 처리 가능: 개별 신고 클릭 시 통합 상세 페이지로 이동하여 사진, 위치, 사용자 정보, 처리 기록 등을 확인하고, 조치 상태를 변경하거나 메모를 남길 수 있다.
- CCTV 연동 모니터링: CCTV 또는 영상 분석 시스템과 연동된 경우, 특정 지역의 실시간 화면을 확인하여 위험 상황을 사전에 파악할 수 있다.

설계 의도

신고 발생 → 확인 → 조치 → 기록의 관리자 업무 흐름을 단일 인터페이스로 통합하며, 실시간 데이터 반영을 통해 현장 대응 속도 및 정확도를 높이고, 장기적인 데이터 기반 의사결정을 지원한다.

25

CCTV 관리 화면	
	
CCTV 관리 화면은 관리자에게 실시간 영상 기반 출몰 감지, 카메라 제어, 상황 확인 기능을 제공하는 모니터링 전용 화면이다. 여러 CCTV를 동시에 관리할 수 있도록 리스트, 실시간 영상, 감지 결과를 직관적으로 배치하여 빠른 상황 파악이 가능하도록 구성되었다.	
주요 기능	
- AI 감지 결과 표시: YOLOv5 기반 분석이 활성화된 경우, 화면에서 감지된 동물 종류, 신뢰도를 실시간으로 표시하여 관리자가 위험 상황 여부를 즉시 판단할 수 있다. - 카메라 선택 및 전환: 여러 대의 CCTV 목록을 제공하여 관리자가 원하는 카메라로 즉시 전환할 수 있고, 특정 지역의 영상을 집중적으로 모니터링할 수 있다.	
설계 의도	
현장 상황을 빠르게 파악하고 즉각적 대응을 가능하게 하기 위한 화면으로 AI 감지 결과를 시각적으로 표시해 관리자의 판단 부담을 줄이고 오탐지와 누락을 최소화할 수 있다.	

신고 처리 및 관리 화면

관리자 페이지
SENCITY
사이트 바로가기
사이토 관리
대시보드
CCTV 관리
신고 처리 및 관리
설정
사용자 관리
문헌도 및 통계
데이터 분석 및 통계
콘텐츠 관리
공지 설정

신고 처리 및 관리

총 신고 접수 73
오늘 접수 0
미해결 40

최근 신고 리스트
카테고리(지역)상태신고자
전체 상태
연도-월-일
연도-월-일
검색

ID	동물	사용자	지역	상태
97	멧돼지	yuna	서울특별시 성동구 삼곡이길 200	checking
95	멧돼지	yuna	서울특별시 광진구 아차산로 272	completed
94	왜가리	yuna	서울 광진구 자양동 794-10	completed
70	고라니	한성우	서울 강서구	on_hold
69	족제비	김병찬	부산 해운대구	checking
68	고라니	문지민	경남 진주시	completed
67	멧돼지	김미란	경북 포항시	on_hold
66	족제비	김병찬	인천 연수구	completed
65	왜가리	김은주	경남 진주시	checking

신고한 사용자 TOP
김병찬 4
yuna 3
신태호 3
김미란 2
김은주 2
김한결 2
문지민 2
신기선 2
윤다솔 2
이준 2

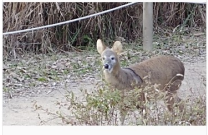
관리자 페이지
SENCITY
사이트 바로가기
사이토 관리
대시보드
CCTV 관리
신고 처리 및 관리
설정
사용자 관리
문헌도 및 통계
데이터 분석 및 통계
콘텐츠 관리
공지 설정

신고 처리 및 관리

총 신고 접수 73
오늘 접수 0
미해결 40

최근 신고 리스트
카테고리(지역)상태신고자
전체 상태
연도-월-일
연도-월-일
검색

ID	동물	사용자	지역	상태
97	멧돼지	yuna	서울특별시 성동구 삼곡이길 200	checking
95	멧돼지	yuna	서울특별시 광진구 아차산로 272	completed
94	왜가리	yuna	서울 광진구 자양동 794-10	completed
70	고라니	한성우	서울 강서구	on_hold
69	족제비	김병찬	부산 해운대구	checking
68	고라니	문지민	경남 진주시	completed
67	멧돼지	김미란	경북 포항시	on_hold
66	족제비	김병찬	인천 연수구	completed
65	왜가리	김은주	경남 진주시	checking



ID 70
동물 고라니
사용자 한성우
지역 서울 강서구
상태 on_hold
일시 2025-09-01 00:00
접수 완료 | 보류 | 답변 완료
(선택) 사용자에게 보낼 메시지/답변

신고 처리 및 관리 화면은 사용자로부터 접수된 야생동물 신고를 실시간으로 확인/분류/처리할 수 있도록 구성된 관리자 전용 화면이다.

주요 기능

- 실시간 신고 목록 표시: 새롭게 접수된 시간순으로 정렬해 한눈에 보여주며, 종·위험도·위치·신고자 정보 등을 즉시 확인할 수 있다.
- 신고 상세 정보 조회: 특정 신고를 클릭하면 사진, 위치, AI 분석 결과 등 상세 정보가 함께 제공되어 현장 상황 파악이 빠르다.
- 처리 상태 변경 가능: 단계별 상태를 관리자가 즉시 변경할 수 있다.

설계 의도

신고 발생 → 조회 → 판단 → 처리의 관리자 업무 흐름을 간소화하며 처리 상태가 사용자 앱에 즉시 반영돼 시민과 관리기관 간의 소통 품질이 향상된다.

27

설정 화면

[illegible]

설정 화면은 관리자 대시보드의 운영 방식을 제어하는 핵심 관리 영역으로, 서비스 정책, 지도 옵션, 알림 방식, 신고 처리 규칙 등 전체 운영 환경을 세부적으로 조정할 수 있는 화면이다.

주요 기능

- 기본 운영 설정: UI/UX 관련 옵션(상태 표시 방식, 메뉴 구조, 배경 정보 등)을 조정하여 관리자 화면 사용성을 최적화한다.
- 지도·위치 설정: 기본 지도 스타일, 표시 레벨, 위치 표시 방식 등을 지정하여 관리자 화면에서 보여지는 지도 정보를 원하는 형태로 구성할 수 있다.
- 알림 관련 설정: 푸시 알림 사용 여부, 신고 발생 시 자동 전송 여부 등 알림 방식을 세부적으로 관리한다.
- 시스템 운영 설정: 로그 보관 기간, API 키 관리, 운영 로그 수집 활성화 등 장기적인 시스템 관리 요소를 통합적으로 조정할 수 있다.

설계 의도

관리자별 운영 스타일과 조직 정책에 맞게 대시보드 환경을 자유롭게 커스터마이징할 수 있으며 지자체, 기관 등 다양한 조직에서도 동일 시스템을 환경에 맞게 조정하여 활용할 수 있는 구조이다.

사용자 관리 화면

관리자 페이지
SENACITY
사이트 바로가기
사이트 관리
대시보드
CCTV 관리
신고 처리 및 관리
설정
사용자 관리
콘텐츠 및 통계
데이터 분석 및 통계
콘텐츠 관리
공지 설정

사용자 관리

이름/이메일 검색

ID	이름	이메일
59	ai_bot	
58	nuri	yunul@gmail.com
57	yuna	ya8030@naver.com
56	jiyeon	sinji@gmail.com
55	신기선	kisun@naver.com
54	나현준	hyunjun@kmail.com
53	김한결	gyeal@daum.net
52	이대훈	daeeab@naver.com
51	신윤철	siver@gmail.com
50	봉욱	wukki@naver.com
49	이성민	sung@hanmail.net
48	곽준우	gkack@naver.com
47	정한영	hanyong@kmail.com

yuna
ya8030@naver.com

개요 | 활동 | 보안

후 신고
3
전역 신고 누계

최근 30일 신고
2
최근 30일

평균 체면
0
현재 로그인 수

프로필
가입일
2025-08-29
최근 로그인
2025-11-14 01:35
그룹

이메일 인증
상태
미인증

관리자 페이지
SENACITY
사이트 바로가기
사이트 관리
대시보드
CCTV 관리
신고 처리 및 관리
설정
사용자 관리
콘텐츠 및 통계
데이터 분석 및 통계
콘텐츠 관리
공지 설정

사용자 관리

이름/이메일 검색

ID	이름	이메일
59	ai_bot	
58	nuri	yunul@gmail.com
57	yuna	ya8030@naver.com
56	jiyeon	sinji@gmail.com
55	신기선	kisun@naver.com
54	나현준	hyunjun@kmail.com
53	김한결	gyeal@daum.net
52	이대훈	daeeab@naver.com
51	신윤철	siver@gmail.com
50	봉욱	wukki@naver.com
49	이성민	sung@hanmail.net
48	곽준우	gkack@naver.com

yuna
ya8030@naver.com

개요 | **활동** | 보안

활동 Top3
옛대지 1
옛도끼 1
재가리 1

지역 Top3
서울특별시 성동구 마조 2호1가길 3 1
불광천 1
서울 광진구 자양동 1794-10 1

최근 신고 5건
#99 - 《재무 없음》
옛도끼 - checking - 2025-11-24 05:53
서울특별시 성동구 마조로1가길 3
#98 - 《재무 없음》
황샘은 - checking - 2025-11-24 05:52
서울특별시 성동구 마조로1가길 3
#95 - 《재무 없음》
옛대지 - completed - 2025-11-19 07:17
불광천
#94 - 《재무 없음》
재가리 - completed - 2025-10-14 17:21
서울 광진구 자양동 794-10

사용자 관리 화면은 관리자 대시보드에서 앱 사용자와 관리자 계정을 통합적으로 조회/검색/관리할 수 있도록 구성된 화면이다. 사용자 정보와 활동 기록을 효율적으로 관리해 전체 서비스 운영에 필요한 기초 데이터를 제공한다.

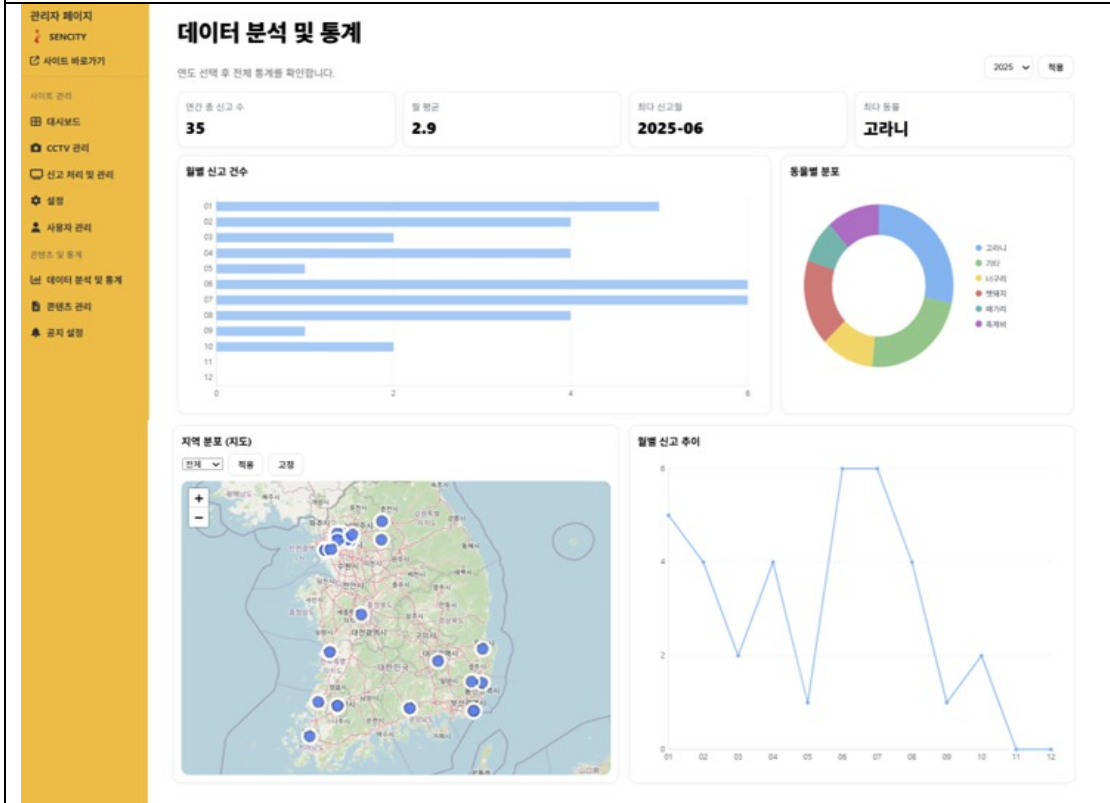
주요 기능

- 사용자 리스트 조회: 사용자 ID, 이름, 이메일, 가입일 등 기본 정보를 표 형태로 정렬해 제공하며, 전체 사용자 현황을 한눈에 파악할 수 있다.
- 사용자 상세 정보 확인: 특정 사용자를 클릭하면 신고 내역, 최근 활동, 계정 상태 등을 상세하게 확인할 수 있어 문제 발생 시 즉시 대응이 가능하다
- 계정 관리 기능: 필요 시 계정 비활성화, 권한 설정 등 관리 조치가 가능해 안전하고 안정적인 서비스 운영을 지원한다.

설계 의도

사용자 활동 기록 기반의 신속한 대응 및 모니터링 체계를 구축하며 데이터 기반 운영을 위해 사용자 정보를 정확하게 파악하는 기초 관리 기능을 제공한다.

데이터 분석 및 통계 화면



데이터 분석 및 통계 화면은 사용자 신고와 AI 감지 데이터를 종합해 도시 내 야생동물 출몰 패턴을 시각적으로 분석할 수 있도록 구성된 관리자용 화면이다.

주요 기능

- 기간별 통계 요약(KPI 카드): 핵심 지표를 요약해 전체 흐름을 빠르게 파악할 수 있다.
- 종별 출몰 비율 분석(도넛차트): 특정 기간 동안 어떤 야생동물이 가장 많이 발견되었는지 비율로 확인할 수 있다.
- 지역별 발생빈도(막대/선 그래프): 신고 발생이 많은 특정 시기를 시각화하여 출몰 경향을 쉽게 파악할 수 있다.
- 지도 기반 출몰 분포(지도 시각화): 지역별 밀집 정도를 직관적으로 확인할 수 있으며, 위험 지역을 식별하는 데 활용된다.

설계 의도

복잡한 데이터를 시각화 중심으로 구성하므로 출몰 패턴 분석 후 정책 결정에 직접 활용 가능하다. 또한 AI 감지 데이터와 사용자 신고 데이터를 통합해 신뢰도 높은 분석 체계를 구성한다.

콘텐츠 관리 화면*

관리자 페이지

SENCITY

사이트 바로가기

사이트 관리

대시보드

CCTV 관리

신고 처리 및 관리

설정

사용자 관리

알림 및 통계

공지 설정

콘텐츠 템플릿

데이터 분석 및 통계

공공데이터 활용

콘텐츠 템플릿

제목/태그 검색

검색

템플릿 선택

템플릿 추가

새 콘텐츠

추천 템플릿

주간 통계 하이라이트

야생동물 알람 카드

알 배너 공지

엔베 노출 중인 배너

최근 생성

콘텐츠 관리 화면은 관리자 대시보드에서 앱 내에 노출되는 배너·이벤트·안전 안내·추천 정보 등을 직접 생성·수정·배포할 수 있는 영역이다. 실시간으로 콘텐츠를 업데이트할 수 있어 사용자에게 최신 정보와 안전 알리를 지속적으로 제공할 수 있다.

통계 그래프 개요 상세 목표 그래프 종류 막대 그래프 선 그래프 면적 그래프 파이 차트 히트맵 대위터 통계 통계 그래프 통계 그래프 통계 그래프 통계 그래프	통계 그래프 화면은 축적된 야생동물 출몰 데이터를 다양한 시각화 형태로 분석하고, 관리자 대시보드에 배치할 통계 화면을 구성하기 위해 사용된다. 막대 그래프, 선 그래프, 파이 차트, 히트맵 등 여러 그래프 유형을 선택할 수 있으며 월별 신고 건수와 종별 분포 같은 데이터 종류에 따라 적합한 시각화 방식을 적용할 수 있다.
---	--

주요 기능
- 콘텐츠 생성·수정·배포: 앱 화면에 표시되는 배너 등을 관리자가 직접 등록, 배포한다. - 추천 템플릿 제공: 자주 사용하는 콘텐츠 유형(안전 수칙, 위험 지역 안내 등)을 템플릿 형태로 제공하여 빠르게 생성 가능하다. - 배포 내역 및 관리 기능: 작성한 콘텐츠의 등록 일시, 노출 기간, 게시 상태 등을 리스트로 관리하며 즉시 수정/삭제/재배포가 가능하다. - 지도 기반 안전 알림 표시 연동: 특정 지역의 위험 안내 콘텐츠는 지도 화면에도 실시간 반영되어 사용자에게 중요 정보를 빠르게 전달한다.
설계 의도
관리자가 직접 콘텐츠를 업데이트함으로써 실시간 안전 정보 제공 체계를 강화하고 템플릿 기반 작성으로 관리 부담 감소 및 콘텐츠 품질 일관성을 유지할 수 있다. 또한, 관리자가 수동으로 작성하고 배포하기 때문에 AI 기반 반자동 형태로 신뢰성을 확보할 수 있다.

관리자 페이지

SENCITY

사이드 바로가기

사이트 관리

대시보드

CCTV 관리

신고 처리 및 관리

설정

사용자 관리

콘텐츠 및 통계

데이터 분석 및 통계

콘텐츠 관리

공지 설정

공지 설정

다 선택도감전체

제목(본문 검색(그룹 공지 위주))

검색

선택 공지 무시 전송

ID	제목	대상	유형	생성 시각
76	상태 변경 안내	개인	개인	2025-10-29T14:43:40.667807+09
신고 완료되었습니다. 안전한 곳으로 대피하세요				
66	서비스 점검 안내	전체	그룹	2025-10-25T11:56:18.557140+09
오늘 22:00~23:00 점검 예정입니다.				
38	상태 변경 안내	개인	개인	2025-09-04T00:00:00+09:00
고려하는 서울 강서구 한강변 한강 확인 용이해 구조 대원을 중배 중입니다.				
58	주의/위험 안내	전체	그룹	2025-09-03T23:12:38+09:00
산길 낙석 및 미끄럼 주의 위험 구간에서는 주변을 잘 살피고 조심하세요.				
57	환경 보호 안내	전체	그룹	2025-09-03T23:12:36+09:00
56	신고 급중 주의	전체	그룹	2025-09-03T23:12:34+09:00
55	산불 주의 안내	전체	그룹	2025-09-03T23:12:32+09:00

공지 설정 화면은 관리자 대시보드에서 앱 사용자에게 전달되는 전체 공지, 안내, 위험 알림 등을 직접 작성하고 관리할 수 있는 기능을 제공하는 화면이다.

선택 공지 무시 전송

제목

주의/위험 안내

본문

산길 낙석 및 미끄럼 주의 위험 구간에서는 주변을 잘 살피고 조심하세요.

대상 옵션 (기본: 전체)

특정 사용자 ID 침표로 입력 (예: 12,34,56)

비우면 전체 발송

취소 전송

공지 내용과 노출 조건을 세부적으로 설정하여 사용자에게 필요한 정보를 정확한 시점에 전달할 수 있다.

주요 기능

- 공지 목록 조회: 공지 제목, 대상, 유형, 생성 시각 등을 리스트로 확인하여 전체 공지 현황을 한눈에 파악할 수 있다.

- 공지 작성 및 수정: 제목·본문·노출 대상/기간 등을 입력해 새로운 공지를 생성할 수 있으며, 기존 공지는 클릭하여 내용 수정 및 재배포가 가능하다.

설계 의도

관리자가 모든 공지를 일관된 형식으로 체계적으로 관리함으로써 빠른 공지 작성·수정·배포로 위기 상황에서도 신속한 정보 전달 체계 구축이 가능하다.

공공데이터 활용 화면

관리자 페이지

SENCITY

사이트 바로가기

사이트 관리

다시보드로

CCTV 관리

신고 처리 및 관리

설정

사용자 관리

콘텐츠 및 통계

공지 설정

콘텐츠 관리

뉴스 데이터 분석 및 통계

공공데이터 활용

공공데이터 활용

실시간 야생동물 출현 데이터를 공공기관에서 조화 분석할 수 있는 화면입니다.

동물 정보 요약

동물명	코드	분류 그룹	주요사항
개	Dog	반려/도심	유기견 또는 공격성이 있는 개체일 수 있으니 함부로 접근하지 마세요. 필요 시 경찰 기관에 신고하세요.
고라니	Goat	중형 포유류	놀러온 급하게 뛰어들 수 있어 충돌 위험이 있습니다. 가까이 가지 말고 조용히 거리를 유지하세요.
고양이	Cat	반려/도심	칼집양이는 경계심이 강할 수 있습니다. 무리한 접촉을 피하고, 다친 개체는 경찰 기관에 신고하세요.
너구리	Raccoon	중형 포유류	사람에게 익숙해 보일 수 있으나 접촉을 피하세요. 먹이를 주지 말고 거리를 유지하세요.
노루	Roe_deer	중형 포유류	도로로 갑자기 뛰어들 수 있어 안전차 주의가 필요합니다. 발견 시 주변에 알리고 접근하지 마세요.
다람쥐	Squirrel	소형 포유류	직접 위험은 낮지만 물릴 수 있습니다. 먹이 제공은 삼가고, 관찰만 하세요.
멧돼지	Wild_boar	대형 포유류	위험 시 공격될 수 있어 매우 위험합니다. 즉시 안전한 곳으로 이동하고 경찰 기관에 신고하세요.
멧토끼	Hare	소형 포유류	놀러온 급하게 이동하므로 추적하지 마세요. 가까이 접근하지 말고 관찰만 하세요.
반달가슴곰	Asiatic_...	대형 포유류	매우 위험한 야생동물입니다. 절대 접근하지 말고 즉시 신고하세요. 사전 촬영을 위해 가까이 가지 마세요.
왜가리	Heron	조류	직접 위험은 낮으나 동자새가 주변은 예민할 수 있습니다. 가까이 접근하지 마세요.
족제비	Weasel	중형 포유류	위험을 느끼면 공격하거나 물 수 있습니다. 접촉을 피하고 안전거리를 유지하세요.
큰다람쥐	Great_e...	조류	위험성은 낮지만 서식지 주변에서는 방해될 수 있습니다. 가까이 다가가지나 쫓지 마세요.
청대뿔	Chipmunc...	소형 포유류	물릴 수 있으나 손으로 만지지 마세요. 먹이 제공은 삼가고 주변 위험을 유지하세요.

공공데이터 활용 화면은 공공기관 및 내부 시스템에서 관리되는 야생동물 정보를 정리하여 제공하는 데이터 조회용 페이지로, 동물별 기본 정보와 주의사항을 함께 확인할 수 있도록 구성되었다.



공공데이터 활용 페이지의 일부 그래프(월별 출몰 추세, 신고 데이터 요약)이다. 연도 기준으로 동물별 출현 빈도와 월별 출몰 추이를 막대 그래프로 비교할 수 있으며 하단 표를 통해 수치 정보를 함께 확인할 수 있다. 이를 통해 관리자는 자주 출현하는 종과 계절별 위험 시점을 빠르게 파악할 수 있고 예방 활동 계획, 인력 배치, 경고 알림 강화 등 데이터 기반 대응 전략 수립에 활용할 수 있다.

주요 기능

- 공공 및 내부 데이터를 체계적으로 정리하여 관리자와 관련 기관이 동일한 기준의 정보를 참고할 수 있도록 지원한다.
- 동물명, 분류 코드, 생물학적 분류 그룹, 행동 특성, 주의사항과 같은 핵심 정보를 표 형태로 제공
→ 출몰 상황 발생 시 대응 기준을 빠르게 확인하고 일관된 안내와 조치를 가능하게 한다.
- 해당 정보는 알림 문구나 콘텐츠를 구성할 때 참고 자료로 활용될 수 있어 정보 전달의 정확성과 신뢰도를 높이는 역할을 수행한다.

설계 의도

출몰 데이터와 행동 특성 정보를 함께 제공함으로써 실제 대응에 즉시 활용 가능한 정보 화면을 구현하는 데 있다. 이를 통해 관리자는 통계 화면이나 탐지 결과를 해석할 때 공공데이터를 근거로 위험도를 판단할 수 있으며, 지자체와의 협업 과정에서도 동일한 정보 기준을 공유할 수 있다.

4. 서비스 이용 흐름

SENCITY의 전체 서비스는 자동 감지 → AI 분석 → 데이터 저장 → 실시간 알림 → 관리자 대응 → 사후 분석으로 이어지는 순환 구조로 설계되어 있다. 모든 단계는 자동화와 실시간성을 중심으로 구성되어, 사용자와 관리 기관 모두가 즉각적으로 상황을 파악하고 필요한 조치를 수행할 수 있다.

1. 영상 수집 단계

- RTSP 기반 웹캠 또는 ESP32-CAM 모듈이 실시간 영상 촬영
- 촬영된 영상/이미지는 Django 백엔드로 전송

2. AI 분석 단계

- YOLOv5s 모델이 객체를 탐지하고 Bounding Box 생성
- 탐지된 객체를 EfficientNet-B1 모델이 10종 동물로 분류
- 신뢰도 80% 이상일 경우 '야생동물 출몰'로 판정

3. 데이터 저장 단계

- Django ORM을 통해 SQLite(DB)로 자동 저장
- 저장되는 항목
 - 동물 종류
 - 감지 시각
 - 위치 정보(GPS)
 - 신뢰도
 - 감지 이미지

4. 알림 발송 단계

- 출몰 위치 기준 반경 2km 이내 사용자 자동 조회
- Firebase Cloud Messaging(FCM)을 통해 푸시 알림 발송
- 알림 예: "고라니가 ○○동 인근에서 목격되었습니다. 주의하세요!"

5. 모니터링 및 관리자 대응

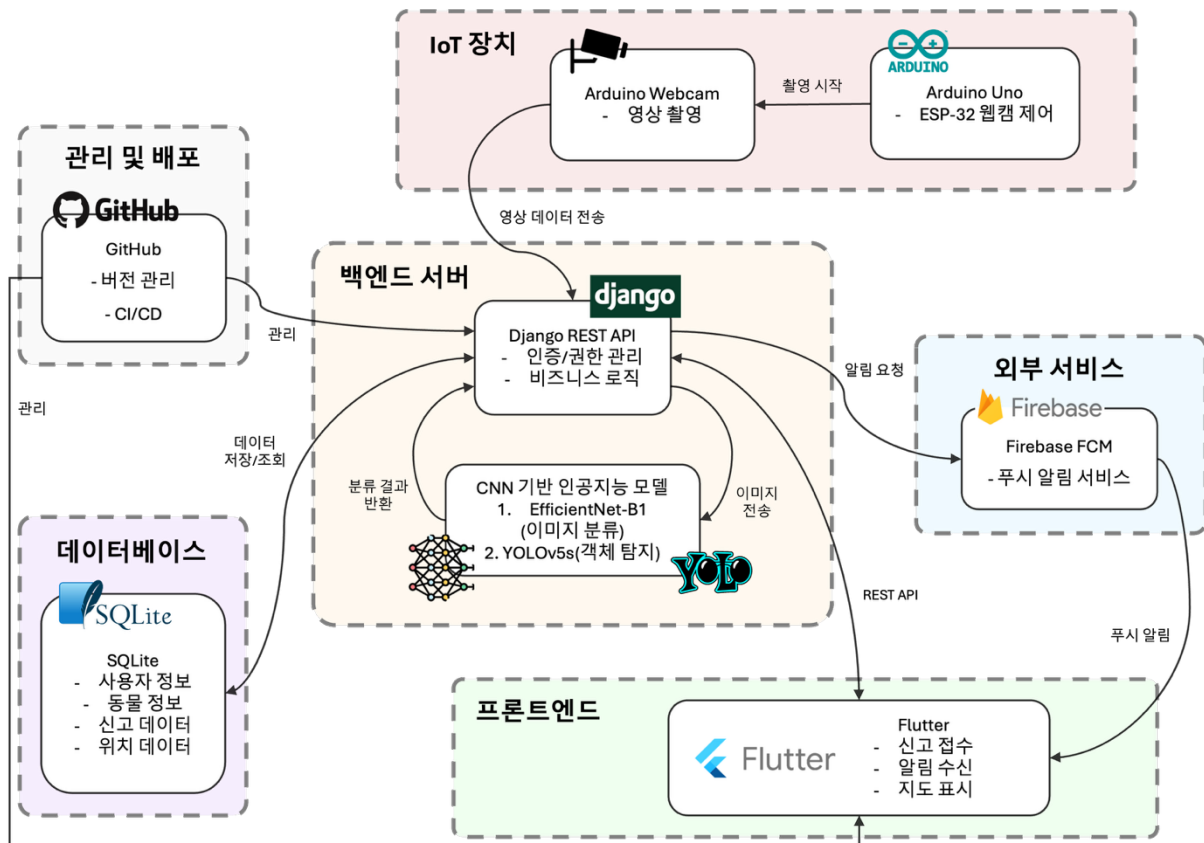
- 관리자 웹 대시보드에 실시간 출몰 정보 표시
- 필요 시 현장 출동 요청, 지역 안내 방송, 또는 추가 확인 수행

6. 사후 분석 단계

- 축적된 출몰 데이터를 기반으로 지역/시간대별 통계 분석
- 주간/월간 리포트 생성
- 야생동물 관리 및 정책 개선에 활용

5. 시스템 설계 및 기술 아키텍처

1. 전체 시스템 구성도



본 시스템은 IoT 장치, 백엔드 서버, AI 분석 모듈, 모바일 프론트엔드, 외부 서비스(FCM), 데이터베이스 및 버전 관리 환경으로 구성되어 있으며, 모든 요소는 Django REST API 중심으로 통합 운영된다. 각 구성 요소의 역할은 다음과 같다.

1) 사용자 모바일 애플리케이션(Flutter)

사용자 행동이 시작되는 입력 지점이자, 위험 정보를 직접 제공받는 출력 지점으로 동작

- 신고 제출 및 GPS 기반 위치 자동 인식
- AI 판별 결과 확인
- 지도 기반 출몰 정보 조회
- 관리자 공지 및 위험 알림(FCM) 실시간 수신

- 2) 백엔드 서버(Django REST Framework): 전체 기능을 연결하는 중앙 허브
 - 사용자 인증 처리(JWT)
 - 신고·탐지 데이터 저장
 - AI 분석 요청 처리
 - 대시보드 데이터 제공
 - 알림 발송 트리거 생성

- 3) AI 분석 모듈(EfficientNet-B1 / YOLOv5s)

자동 분석으로 오인 신고 감소, 실시간 위험 감지, 정책 기반 데이터 추적

 - EfficientNet-B1: 사용자 신고 이미지 기반 동물 종 분류
 - YOLOv5s: CCTV 영상에서 야생동물 객체 탐지
 - 탐지 결과를 서버로 전달하여 실시간 정보 반영

- 4) 데이터베이스(SQLite): 단일 스키마로 데이터 관리하여 일관성 유지
 - Users / Reports / Animal / Locations / Feedbacks / DetectionEvent / Notification / DeviceToken 등 구조적 저장

- 5) 관리자용 웹 대시보드(Django Template 기반)
 - 신고·탐지 실시간 모니터
 - 신고 처리 상태 변경
 - 공지·알림 위험 생성 및 배포
 - 사용자 관리 및 데이터 분석

- 6) FCM 푸시 알림 서버
 - 고위험 탐지 시 관리자가 대시보드에서 즉시 사용자 스마트폰으로 알림 전달

2. 백엔드 아키텍처

백엔드는 모바일 앱-AI 모듈-대시보드-알림 서버를 연결하는 중심 레이어로, 모든 기능이 REST API로 제공된다. 전체 시스템은 API Layer - Service Layer - Database Layer - AI Integration Layer로 나뉘어져 있으며 기능별 처리 구조를 명확히 분리한 계층형 구조로 설계하였다.

각 API는 인증/데이터 검증/비즈니스 로직 수행 단계를 분리하여 처리되며, 모바일 앱, 대시보드, AI 모듈이 동일한 구조로 서버에 접근할 수 있도록 일관된 설계를 따른다.

1) 주요 API 구성

하나의 흐름으로 구조를 연결하는 핵심 기반이며, 서비스 내 모든 기능은 명확히 정의된 REST API를 통해 작동한다. API 구성으로는 사용자 영역과 관리자 영역이 있으며 다음과 같은 특징을 갖는다.

- 사용자 영역: 모바일 앱에서 사용하는 API로, 회원가입·로그인·신고 제출·지도 데이터·기록 조회·마이페이지 수정 등 앱의 모든 기능을 실제로 작동시키는 핵심 API

<u>SC-01</u>	<u>회원가입</u>	POST	/api/users/signup	신규 사용자 계정 생성
<u>SC-02</u>	<u>로그인</u>	POST	/api/users/login	사용자 로그인 및 JWT 발급
<u>SC-03</u>	<u>홈</u>	GET	/api/notifications/latest	출현 알림 및 통계 기록 출력
<u>SC-04</u>	<u>기록 조회</u>	GET	/api/reports	사용자의 신고 내역 조회

이를 통해 사용자는 앱에서 신고 → 지도 확인 → 기록 조회 → 알림 수신까지의 전 과정을 API를 통해 수행할 수 있다.

- 관리자 영역: 웹 대시보드에서 신고 조회·상태 변경·사용자 관리·데이터 분석에 활용

<u>AD-01</u>	<u>관리자 페이지</u>	GET	/api/admin/reports	서버 내 데이터 조회
<u>AD-02</u>	<u>관리자 페이지</u>	PUT	/api/admin/reports/{id}	신고 상태 변경

이를 통해 실시간 신고 관리, 위험 알림 전송, CCTV 탐지 모니터링을 수행한다.

서버 내부의 흐름은 자동화된 로직으로 구성되어 있다. 서버에서 처리된 데이터는 Django ORM 기반 구조에 통합 저장되며, 하나의 데이터 모델에서 연결되어 시각화·통계 분석·알림에 바로 활용될 수 있도록 구성되어 있다.

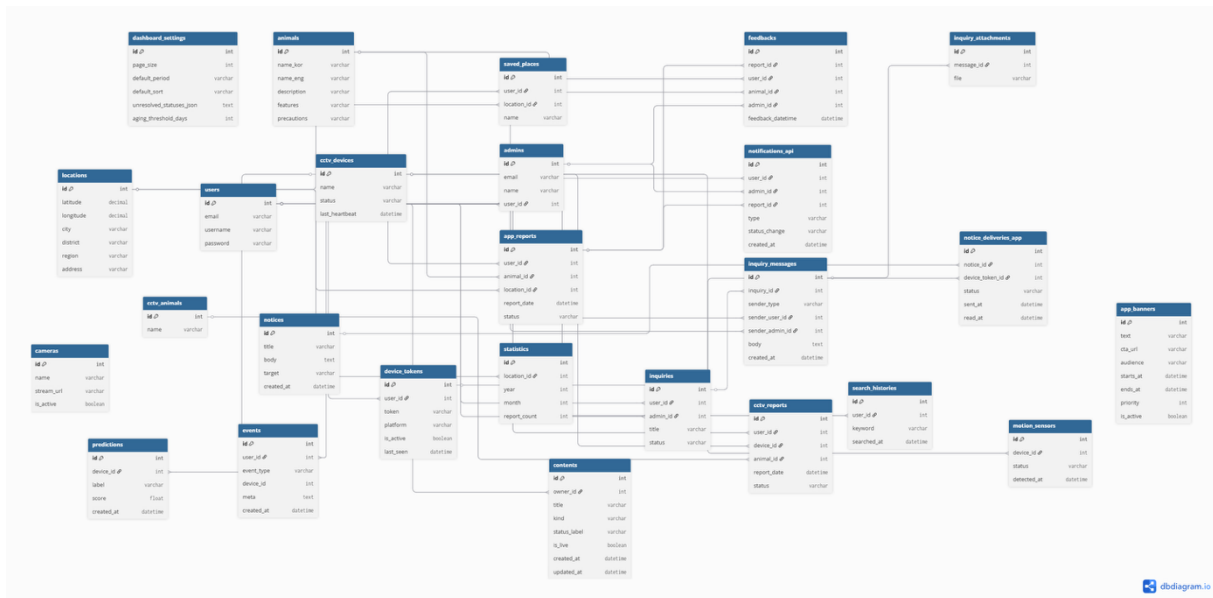
3. AI 모델 구조

SENCITY의 AI 분석 모듈은 이미지 기반 종 분류와 CCTV 영상 기반 객체 탐지의 이중 구조로 구성되어 있다. 두 모델은 Django 서버와 직접 연동되어 자동화하며, 사람이 지속적으로 관찰하지 않아도 실시간 위험 감지가 가능해지는 것이 핵심이다.

서버 내부에서 각각 독립적으로 동작하지만, 최종적으로 하나의 AI 분석 파이프라인으로 통합되어 속도, 정확도, 운영 편의성을 충족한다.

	EfficientNet-B1 (이미지 분류)	YOLOv5s (객체 탐지)
역할	동물 종 자동 분류	CCTV 영상에서 객체 실시간 탐지
입력 데이터	사용자 촬영 사진	CCTV 영상 프레임(실시간 스트림)
출력 결과	예측된 종(Label), 신뢰도	bounding box, 클래스, 신뢰도
사용 목적	신고 정확도 향상, 오인 신고 감소	위험 상황 즉시 확인 및 자동 감지
주요 강점	경량 모델 대비 높은 정확도 모바일 환경에 적합 사진 기반 신고 자동화	빠른 탐지 속도 다중 객체 탐지 가능 자동 모니터링
결과 활용	Reports 테이블에 저장 → 지도 및 통계 반영	DetectionEvent 저장 → 대시보드 실시간 표시
알림 연동	고위험 종 발견 시 FCM 알림	위험 지역 CCTV 탐지 시 즉시 알림

4. 데이터베이스 설계



서비스 전반의 데이터를 일관된 구조로 관리하기 위해 구성되었다. SQLite 기반으로 운영되며 ERD 핵심 축은 다음 세 가지 연계 구조이다.

1) 사용자(Users) 중심 구조

Users 엔터티는 신고-알림-기기 토큰 등 모든 핵심 기능과 연결되는 중심 노드이다. 한 사용자는 여러 건의 신고를 생성할 수 있으며, 알림과 기기 정보도 Users와 직접 연결된다.

2) 신고(Reports) — 동물(Animal) — 위치(Locations)의 연결

AI 분석 결과와 사용자 신고를 통합하기 위해 아래 구조를 사용한다.

- Reports: 이미지, 좌표, 시간, 상태 등 신고의 본문 데이터
- Animal: EfficientNet-B1이 판별한 9종 분류 결과를 표준화하여 저장
- Locations: GPS 좌표 기반의 위치 저장 엔터티

3) CCTV 기반 탐지(Prediction, MotionSensor, CCTVDevice)

센서 감지 → 영상 프레임 → YOLO 분석 → DetectionEvent 저장의 자동화 파이프라인을 반영했다.

- CCTVDevice: 실제 현장에 설치된 카메라 장치 정보
- MotionSensor: PIR 모션 센서 등 카메라 주변 동작 감지 장치
- Prediction: YOLOv5 분석 결과(이미지, 라벨)를 기록

5. 데이터 흐름 구조

데이터 흐름은 입력 → 처리 → 출력의 단일 파이프라인으로 구성되며, 사용자 신고와 CCTV 탐지 데이터가 동일한 구조 안에서 통합 처리되도록 설계되었다.

1) 입력 단계(Input Layer): 데이터는 다음 세 경로에서 발생한다.

- 사용자 신고(모바일 앱): 촬영 이미지, 위치 정보, 신고 메타데이터
- CCTV 자동 탐지(YOLOv5s): 영상 프레임 기반 탐지 이벤트
- 모션 센서 감지(MotionSensor): 움직임 감지 신호

→ 모든 입력은 Django REST 서버로 전달된다.

2) 처리 단계(Processing Layer): 서버는 입력 데이터 유형에 따라 다음 처리를 수행한다.

- 신고 데이터 저장 및 EfficientNet-B1 분석 요청
- 영상 기반 탐지 결과 저장(Prediction / DetectionEvent)
- 데이터 정제 및 결합(동물 종, 좌표, 시간)
- 관리자 대시보드 실시간 반영

→ 서버는 AI 분석 모듈과 연동된 단일 처리 엔진 역할을 한다.

3) 저장 단계: 모든 데이터는 SQLite 기반의 단일 스키마 안에 저장된다.

- 사용자(Users)
- 신고(Reports)
- 알림(Notifications)
- 동물(Animal)
- 위치(Locations)

→ 신고-탐지-알림이 하나의 통합 데이터 모델 내에서 연결된다.

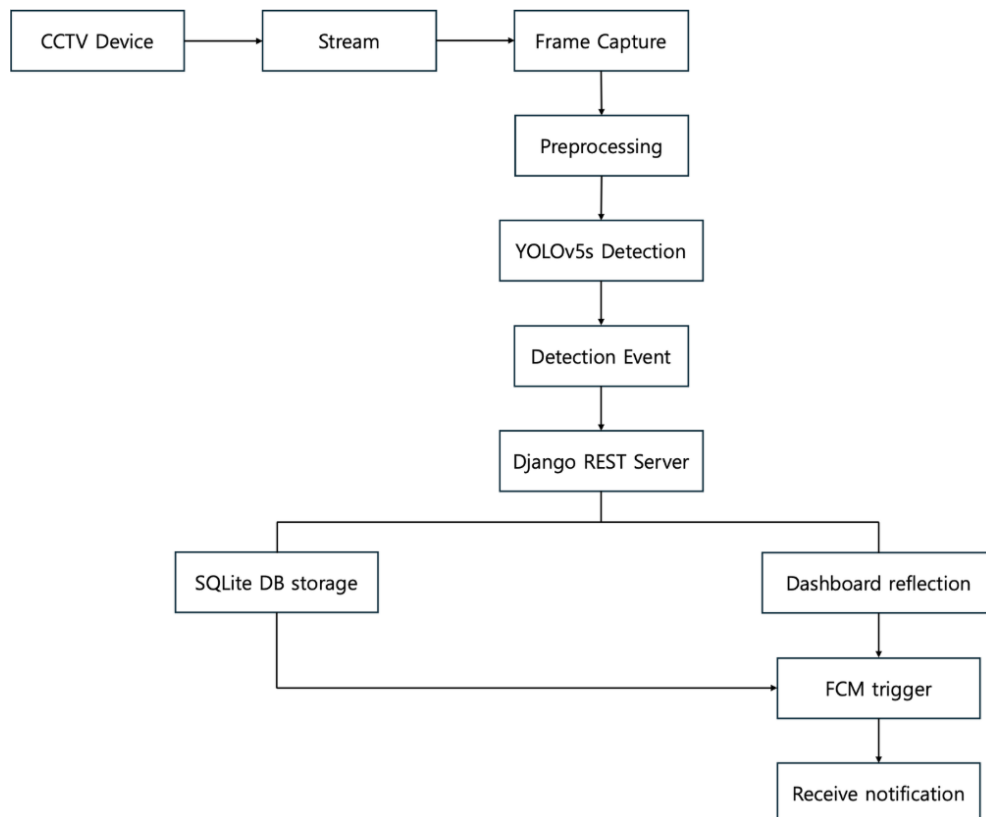
4) 출력 단계(Output Layer): 저장된 데이터는 두 채널로 전달된다.

- 관리자 대시보드: 신고 목록, 분석 결과, CCTV 탐지 현황, 사용자 알림 내역
- 모바일 사용자: 위험 알림(FCM), 신고 결과 반영, 지도 기반 출몰 정보

6. 영상 처리 구조

CCTV 모듈은 영상 스트림 → 전처리 → YOLOv5s 탐지 → 이벤트 저장 → 알림 전송으로 이어지는 자동화 파이프라인으로 구성된다. 사람의 개입 없이 24시간 동작하며, 탐지 결과는 즉시 서버와 대시보드에 반영된다.

- 1) 전처리 단계: 실시간으로 수집된 프레임은 모델 입력에 맞게 전처리된다.
- 2) 이미지 리사이징, 텐서 변환(정규화 포함), 프레임 선택 등 전처리를 통해 AI 분석 속도와 정확도를 동시에 확보한다.
- 3) 객체 탐지 단계: 전처리된 프레임은 다음 정보를 산출한다.
바운딩 박스, 탐지된 동물 클래스 이름, 모델 신뢰도가 산출되며 Confidence가 설계된 임계치(0.7)를 넘으면 탐지 이벤트로 인정된다.
- 4) 서버 반영 단계: DB에 저장된 이벤트는 관리자 대시보드로 업데이트되며, 고위험 종(멧돼지, 곰) 또는 사용자 위험 지역 인근 탐지 시 위험 알림 트리거를 생성하여 FCM을 통해 시민에게 경고 메시지를 전달한다.



6. AI 모델 학습 및 프로젝트 완성도

1. 데이터 수집 및 라벨링

- Google 크롤링

Kaggle 오픈 데이터셋에서 총 10종(고라니, 멧돼지, 다람쥐, 반달가슴곰, 너구리, 멧토끼, 족제비, 왜가리, 강아지, 고양이)의 이미지 데이터 수집 후 부족한 데이터는 Google 크롤링으로 보완하였다. 이후 크롤링으로 얻은 데이터를 수동으로 검수하여 클래스당 500장씩 AI 모델 학습에 적합한 데이터셋을 구축하였다.

crawler.py

```
import os
from icrawler.builtin import GoogleImageCrawler

SPECIES_LIST = [
    ("고라니", "Water deer", "Hydropotes inermis argyropus"),
    ("멧돼지", "Wild boar", "Sus scrofa coreanus"),
    ("다람쥐", "Siberian chipmunk", "Tamias sibiricus"),
    ("너구리", "Raccoon dog", "Nyctereutes procyonoides koreensis"),
    ("반달가슴곰", "Asiatic black bear", "Ursus thibetanus ussuricus"),
    ("멧토끼", "Korean hare", "Lepus coreanus"),
    ("족제비", "Siberian weasel", "Mustela sibirica"),
    ("왜가리", "Grey heron", "Ardea cinerea"),
]

TARGET_NUM = 300

for korean, english, scientific in SPECIES_LIST:
    save_dir = f"images/{korean}"
    os.makedirs(save_dir, exist_ok=True)

    existing_files = [f for f in os.listdir(save_dir) if
                      f.lower().endswith(('.jpg', '.jpeg', '.png'))]
    existing_count = len(existing_files)
    remaining = max(TARGET_NUM - existing_count, 0)

    if remaining == 0:
        print(f"[{korean}] 현재 {TARGET_NUM}장 이상 --> 스킵")
        continue
```

```

print(f"[{korean}] 현재 {existing_count}장 -->
      {remaining}장 크롤링")

crawler = GoogleImageCrawler(storage={"root_dir":
                                     save_dir})
queries = [korean, english, scientific]

per_query = remaining // len(queries) + 1

for query in queries:
    crawler.crawl(keyword=query, max_num=per_query,
                  file_idx_offset='auto')

```

2. 전처리 과정

- EfficientNet-B1

먼저 모든 이미지를 모델 입력 규격에 맞춰 240×240 크기로 리사이즈하였다. 학습 데이터에는 모델의 일반화 성능 향상을 위해 수평 뒤집기, 회전, 밝기·대비 변화 등의 데이터 증강을 적용하였다.

모든 이미지는 텐서 형태로 변환한 뒤 ImageNet 사전학습 모델에 최적화된 평균 및 표준편차 값을 사용하여 정규화를 수행하였다:

- 평균: [0.485, 0.456, 0.406]
- 표준편차: [0.229, 0.224, 0.225]

검증 데이터는 데이터 증강 없이 리사이즈 → 텐서 변환 → 정규화를 적용하여 모델 성능 평가의 일관성을 유지하였다.

efficientnetb1_train.py

```

train_transform =
    transforms.Compose([ transforms.Resize((240, 240)),
                          transforms.RandomHorizontalFlip(),
                          transforms.RandomRotation(15),
                          transforms.ColorJitter(brightness=0.2, contrast=0.2),
                          transforms.ToTensor(),
                          transforms.Normalize([0.485, 0.456, 0.406], [0.229,
                                                                           0.224, 0.225])])

val_transform =
    transforms.Compose([ transforms.Resize((240, 240)),

```

```
transforms.ToTensor(),
transforms.Normalize([0.485, 0.456, 0.406], [0.229,
0.224, 0.225])))
```

- YOLOv5s

1. 데이터 품질 정규화(Quality Normalization)

기존 데이터셋에서는 다양한 촬영 환경과 해상도로 인해 클래스 간 이미지 품질 편차가 존재하였다. 특히 강아지(08_Dog)와 고양이(09_Cat) 클래스는 다른 클래스에 비해 고화질 이미지가 과도하게 많아, 실제 CCTV 환경과의 분포 차이가 크다는 문제가 있었다. 이에 따라 두 클래스를 대상으로 데이터 품질 정규화를 적용하였다.

사용된 알고리즘은 다음과 같다.

- Downscale (0.6~0.8 비율)

실제 저해상도 CCTV 영상의 특성을 반영하여 해상도를 축소함.

- 모션 블러 / 가우시안 블러

야간 CCTV에서 발생하는 흔들림 및 초점 흐림을 재현함.

- ISO Noise / Gaussian Noise

저조도 환경에서 나타나는 센서 노이즈를 시뮬레이션함.

- Brightness & Contrast Shift

야간 촬영, 역광 등 실제 환경의 밝기 변화 반영.

- JPEG Compression (50~75%)

CCTV 영상 전송 과정에서의 압축 아티팩트를 모사함.

이를 통해 특정 클래스만 과도하게 고품질로 학습되는 문제를 해결하여, 실제 환경에 가까운 데이터 분포를 구축하였다.

2. 오버샘플링(Oversampling)

클래스 간 데이터 편차를 줄이기 위해 각 클래스별 이미지 개수를 최소 500장으로 균일화하였다. 이미지 수가 부족한 클래스는 기존 이미지에 데이터 증강을 적용하여 필요한 수만큼 생성하였다.

사용된 증강 기법은 다음과 같다.

- HorizontalFlip(좌우 반전) : 동물의 이동 방향 다양화
- Brightness/Contrast 조절 : 다양한 야외 조도 조건 재현
- Gaussian Noise / Blur : 카메라 품질 및 환경적 영향 반영

- RandomResizedCrop (스케일, 비율 변형): 동물이 화면 내 다양한 위치와 크기로 등장하는 상황 반영
- ShiftScaleRotate: 카메라 기울기, 거리 변화 등을 반영한 기하학적 변환

yolov5s_train.py

```
import os
import cv2
import random
import shutil
from pathlib import Path
from tqdm import tqdm
import albumentations as A

data_root = '/content/data'
original_images_dir = f'{data_root}/images_original'
labels_dir = f'{data_root}/labels'
classes = {
    '00_Goat': 0, '01_Wild boar': 1, '02_Squirrel': 2, '03_Raccoon': 3,
    '04_Asiatic black bear': 4, '05_Hare': 5, '06_Weasel': 6,
    '07_Haron': 7, '08_Dog': 8, '09_Cat': 9
}

def create_cctv_style_augmentation():
    return A.Compose([
        A.Downscale(scale_min=0.6, scale_max=0.8, p=0.5,
interpolation=cv2.INTER_LINEAR),
        A.OneOf([
            A.MotionBlur(blur_limit=5, p=1.0),
            A.GaussianBlur(blur_limit=(3, 5), p=1.0),
        ], p=0.4),
        A.OneOf([
            A.GaussNoise(var_limit=(10, 50), p=1.0),
            A.ISONoise(color_shift=(0.01, 0.03), intensity=(0.1, 0.3), p=1.0),
        ], p=0.4),
        A.RandomBrightnessContrast(
            brightness_limit=(-0.2, 0.1), contrast_limit=(-0.2, 0.1), p=0.5
        ),
        A.ImageCompression(quality_lower=50, quality_upper=75, p=0.4),
    ])

high_quality_classes = ['08_Dog', '09_Cat']
cctv_transform = create_cctv_style_augmentation()

quality_normalized_dir = f'{data_root}/images_quality_normalized'
os.makedirs(quality_normalized_dir, exist_ok=True)

for class_name in classes.keys():
    class_path = os.path.join(original_images_dir, class_name)
    output_class_dir = os.path.join(quality_normalized_dir, class_name)
    os.makedirs(output_class_dir, exist_ok=True)
    if not os.path.exists(class_path):
```



```

        continue
    images = list(Path(class_path).glob('*.*jpg')) +
              W list(Path(class_path).glob('*.*png')) +
              W list(Path(class_path).glob('*.*jpeg'))
    for img_path in tqdm(images, desc=f"{class_name:25s}"):
        img = cv2.imread(str(img_path))
        if img is None:
            continue
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        if class_name in high_quality_classes:
            augmented = cctv_transform(image=img_rgb)
            processed_img = augmented['image']
        else:
            processed_img = img_rgb
        output_path = os.path.join(output_class_dir, img_path.name)
        processed_img_bgr = cv2.cvtColor(processed_img, cv2.COLOR_RGB2BGR)
        cv2.imwrite(output_path, processed_img_bgr)

augmentation_pipeline =
    A.Compose([ A.HorizontalFlip(p=0.5),
                A.RandomBrightnessContrast(brightness_limit=0.2, contrast_limit=0.2, p=0.5),
                A.GaussNoise(var_limit=(10.0, 30.0), p=0.3),
                A.Blur(blur_limit=3, p=0.2),
                A.RandomResizedCrop(size=(640, 640), scale=(0.85, 1.0), ratio=(0.9, 1.1),
p=0.4),
                A.ShiftScaleRotate(shift_limit=0.0625, scale_limit=0.1, rotate_limit=10,
p=0.4),
    ])

TARGET_SAMPLES = 500
oversampled_dir = f'{data_root}/images_oversampled'
os.makedirs(oversampled_dir, exist_ok=True)

for class_name in classes.keys():
    src_class_dir = os.path.join(quality_normalized_dir, class_name)
    dst_class_dir = os.path.join(oversampled_dir, class_name)
    os.makedirs(dst_class_dir, exist_ok=True)
    if not os.path.exists(src_class_dir):
        continue
    src_images = list(Path(src_class_dir).glob('*.*jpg')) +
                  W list(Path(src_class_dir).glob('*.*png')) +
                  W list(Path(src_class_dir).glob('*.*jpeg'))
    for img_path in src_images:
        shutil.copy(str(img_path), dst_class_dir)
    current_count = len(src_images)
    needed = TARGET_SAMPLES - current_count
    if needed <= 0:
        continue
    augmented_count = 0
    with tqdm(total=needed, desc=f"{class_name} 증강") as pbar:
        while augmented_count < needed:
            src_img_path = random.choice(src_images)
            image = cv2.imread(str(src_img_path))

```

```

        if image is None:
            continue
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        augmented = augmentation_pipeline(image=image)
        aug_image = augmented['image']
        new_img_name =
f"{src_img_path.stem}_aug{augmented_count}{src_img_path.suffix}"
        new_img_path = os.path.join(dst_class_dir, new_img_name)
        aug_image_bgr = cv2.cvtColor(aug_image, cv2.COLOR_RGB2BGR)
        cv2.imwrite(new_img_path, aug_image_bgr)
        src_label_path = os.path.join(labels_dir, class_name,
f"{src_img_path.stem}.txt")
        if os.path.exists(src_label_path):
            new_label_path = os.path.join(labels_dir, class_name,
f"{Path(new_img_name).stem}.txt")
            shutil.copy(src_label_path, new_label_path)
        augmented_count += 1
        pbar.update(1)

```

3. 모델 학습 환경 및 파라미터

	EfficientNet-B1 (이미지 분석)	YOLOv5s (객체 탐지)
학습 환경	로컬 워크스테이션 / ASUS GeForce GTX 1650 4GB / TensorFlow 2.x-Keras / Python 3.7+	Google Colab / NVIDIA L4(24GB VRAM) / PyTorch 1.x / AMP 활성화
모델 사양	약 7.8M parameters / 입력 크기 240×240 / CPU 기준 추론 속도 약 50ms	COCO pretrained weights / 입력 크기 640×640
학습 파라미터	Optimizer: Adam(LR 0.001) / Loss: Categorical Crossentropy / Batch 32 / Epoch 23 / EarlyStopping: val_loss 3회 미개선 시 중단	Optimizer: SGD + Cosine LR Scheduler / Batch 32 / Epoch 25 / Image Size 640 / EarlyStopping(patience 8) / Label Smoothing 0.1
데이터 증강	Horizontal Flip / Random Rotation(±15°) / Brightness-Contrast 조정	HSV 조정(H=0.015, S=0.7, V=0.4) / Rotation(±10°) / Translation(±10%) / Scale(±50%) / Horizontal Flip(50%)

4. 학습 결과 및 성능 평가

- EfficientNet-B1

- 정확도 : 88.46%

EfficientNet-B1 모델의 성능 평가는 학습에 사용되지 않은 독립된 검증 데이터셋을 기반으로 수행되었다. 따라서 모델이 이미 본 이미지가 아닌 새로운 이미지에서 실제로 얼마나 잘 예측하는지를 측정한 값이므로 과적합의 영향을 받지 않는다.

또한 모든 검증 이미지에 대해 모델이 예측한 결과(y_pred)와 실제 라벨(y_true)을 1:1로 비교하여 정확도를 계산했기 때문에 전체 데이터에 대해 평균적인 성능을 반영한다.

efficientnetb1_accuracy.py

```
import numpy as np
from tqdm import tqdm

y_true = []
y_pred = []

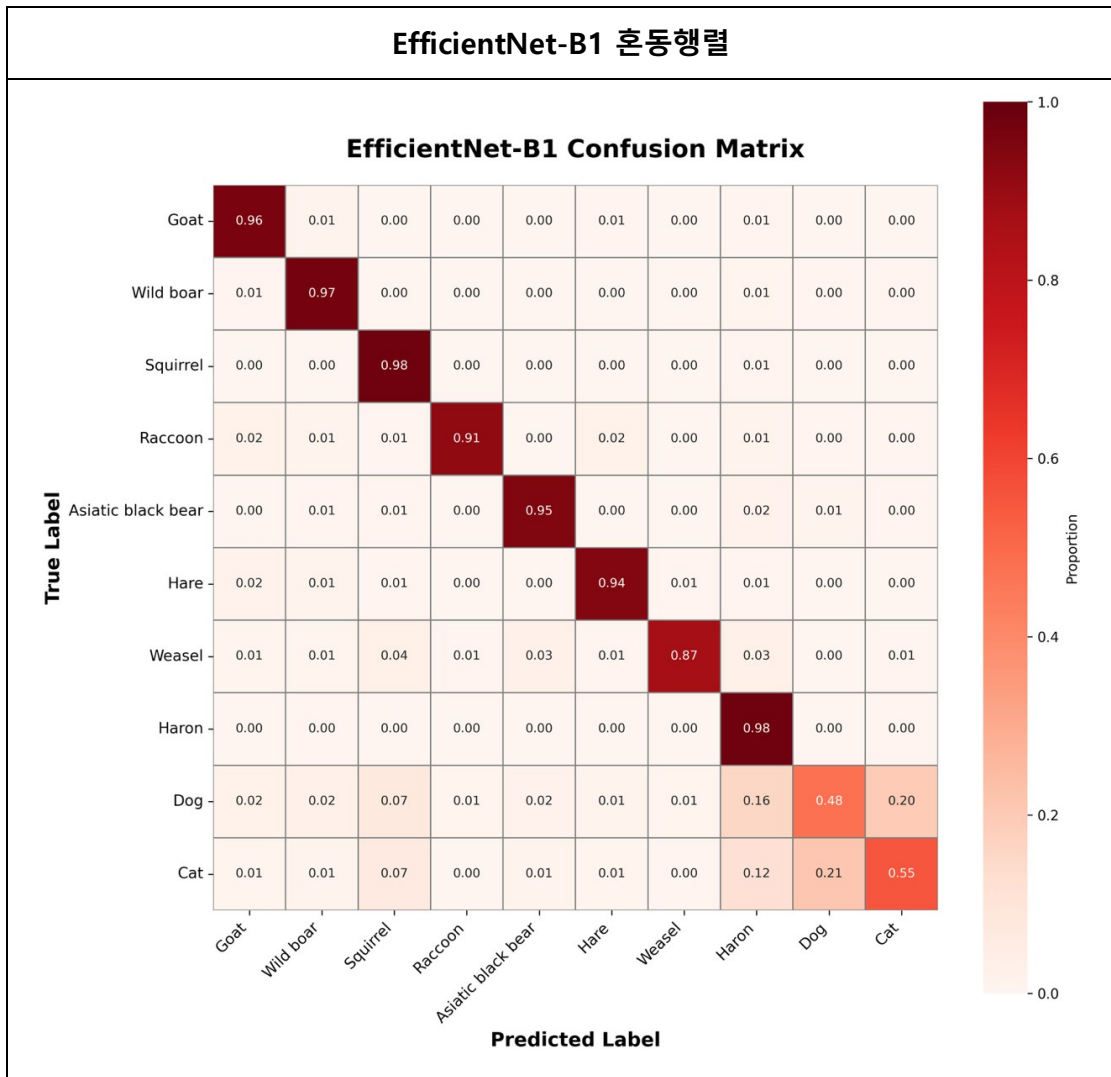
model.eval()
with torch.no_grad():
    with torch.cuda.amp.autocast():
        for images, labels in tqdm(val_loader, desc='Fast Prediction'):
            images = images.to(device, non_blocking=True)

            outputs = model(images)
            _, predicted = torch.max(outputs, 1)

            y_true.extend(labels.numpy())
            y_pred.extend(predicted.cpu().numpy())

y_true = np.array(y_true)
y_pred = np.array(y_pred)
```

```
accuracy = (y_true == y_pred).sum() / len(y_true) * 100
print(f"\n\n전체 정확도: {accuracy:.2f}%")
```



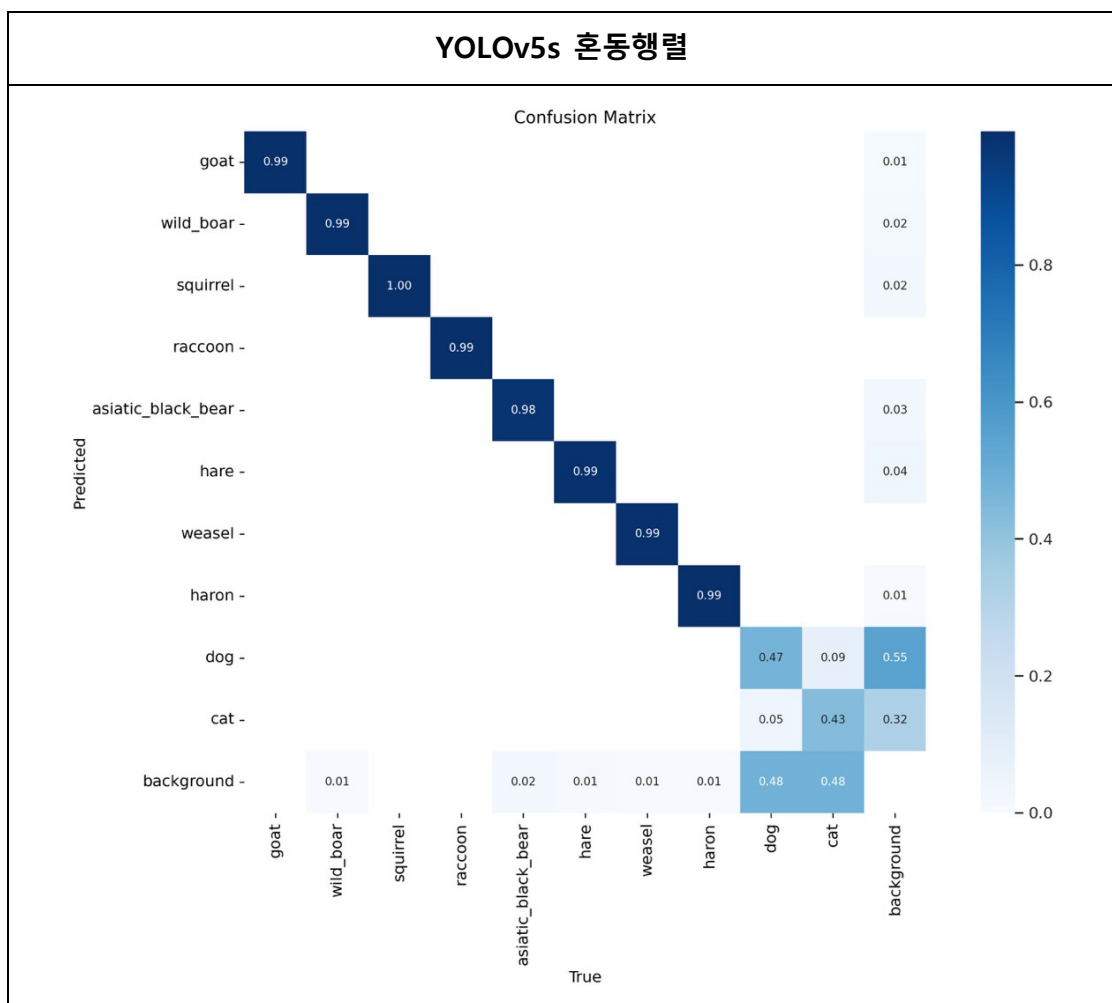
EfficientNet-B1 혼동행렬을 보면, 족제비(Weasel)을 제외한 야생동물 클래스는 재현율 0.90 이상으로 안정적인 성능을 보인 반면 Dog와 Cat 클래스는 재현율이 0.50 내외로 낮다. Dog 클래스의 재현율은 0.48로 절반 이상이 오분류되었고, Cat 클래스 역시 재현율이 0.55로 Dog 및 Heron과 혼동되는 비율이 높았다. 즉, Dog-Cat-Heron 세 클래스 간 특징 구분이 명확하지 않은 것으로 보인다.

추정 원인으로는 색상 유사성이 특징 공간에서 경계를 명확히 학습하는 데 영향을 미친 것으로 보인다. 개선을 위해서는 Dog, Cat, Heron 데이터를 특징이 뚜렷한 이미지 중심으로 보강할 필요가 있다. 또한 데이터 증강을 강화해 다양한 조건에서의 학습을 유도하고, 클래스 가중치 조정이나 손실함수 적용으로 어려운

샘플의 학습을 강화하는 방안이 효과적일 것으로 보인다. 파인튜닝 단계에서는 학습률을 낮추고 에포크를 늘려 세밀한 특징을 학습시키는 것도 필요하다. 반려동물과 특정 야생종 간 오분류가 개선된다면 전체 정확도(88.46%)는 더욱 향상될 것으로 기대된다.

- YOLOv5s

- mAP@50 : 0.889
- Precision : 0.43~1.00
- Recall : 0.43~1.00
- F1-Score : 0.43~1.00



YOLOv5s 모델로 학습한 결과, 고라니, 멧돼지, 다람쥐 등 대부분 야생동물 클래스는 0.98~1.00의 재현율을 보여 야생동물 탐지에 실용적인 성능을 보인다. 하지만 dog, cat, background 클래스는 성능이 낮았다. dog 클래스의 재현율은 0.47로 절반 이상이 background로, 일부는 cat으로 오탐지되었으며 cat 클래스의 재현율은 0.43로 background와 dog로 혼동된다. background는 대부분 dog 또는 cat 클래스로 잘못 탐지되는 문제를 가진다.

이러한 문제의 원인으로는 dog 및 cat 데이터의 배경과의 시각적 유사성, 작은 크기에서 특징 추출의 어려움 등이 있다. 이를 개선하기 위해서는 다양한 환경에서 촬영된 이미지를 확보하고, 배치 크기를 줄여 학습 안정성을 높이는 과정이 필요하다. 또한 dog/cat 클래스의 가중치를 높이거나 앵커 박스를 재설정하는 방법도 고려해야 한다. 전체 클래스에 대한 mAP@50는 0.889로 안정적인 수치를 보이지만 반려동물 구분 성능 개선을 통해 오탐지를 줄일 필요가 있다.

5. 프로젝트 완성도

실시간 시스템 동작 완성도 측면에서 SENCITY는 실제 운영 환경을 고려한 안정적인 통합 파이프라인을 구현하였다. 웹캠을 RTSP URL로 연결하여 실제 CCTV 환경과 동일한 조건에서 테스트를 진행하였으며 Django 서버가 프레임 단위의 스트리밍 영상을 수신한 뒤 YOLOv5s 모델로 전달하여 객체 탐지를 수행하고, 탐지 결과를 저장한 후 모바일 앱으로 실시간 전송하는 전 과정을 정상적으로 동작시키는 데 성공하였다. 이 과정에서 영상 입력부터 탐지, 저장, 알림 전달까지 이어지는 전체 데이터 흐름이 지연 없이 연속적으로 처리됨을 확인하였다.

서버 통신과 데이터 저장 측면에서도 안정성을 검증하였다. Django REST API를 통해 탐지 결과의 종 정보, 발생 시간, 위치 좌표, 신뢰도 값이 실시간으로 SQLite 데이터베이스에 저장되었으며 감지 지연 시간과 데이터 전송 지연, 스트리밍 중단 여부와 같은 핵심 지표를 측정한 결과 실환경에서도 시스템이 문제없이 작동함을 확인하였다. 이를 통해 지속적인 스트리밍 상황에서도 데이터 유실 없이 안정적인 서버 처리가 가능함을 입증하였다.

모바일 앱과 관리자 시스템 구현 수준 또한 실사용을 고려한 완성도를 갖추고 있다. Flutter 기반 사용자 앱에서는 회원가입과 로그인, 지도 기반 위험 지역 표시, 탐지 및 신고 기록 조회, 이미지 신고 기능, FCM 기반 실시간 알림 등 핵심 기능을 모두 구현하였다. 야생동물 탐지 이벤트 발생 시 사용자에게 즉각적인 푸시 알림이 전달되어, 실제 상황에서 신속한 대응이 가능하도록 설계되었다. 관리자 대시보드에서는 탐지 로그의 실시간 조회와 함께 감지된 이미지, 발생 시간, 종 정보 확인, 스트리밍 기반 모니터링 기능을 제공하여 시스템 운영과 관리에 필요한 주요 기능을 모두 포함하였다.

또한 단위 테스트와 통합 테스트를 병행하여 시스템의 신뢰성을 검증하였다. 모델 단위, API 단위, UI 단위로 각각 기능이 정상적으로 동작하는지 확인하였으며 통합 테스트를 통해 전체 데이터 흐름을 점검하였다. 앱 신고 테스트에서는 사용자가 이미지를 업로드하면 Django 서버를 거쳐 EfficientNet-B1 분류 모델이 분석을 수행하고, 결과가 데이터베이스에 저장된 후 다시 앱 알림으로 전달되는 전 과정을 검증하였다. 또한 대시보드 신고 테스트에서는 웹캠에서 입력된 영상이 Django 서버와 YOLOv5s 탐지 모델을 거쳐 데이터베이스에 저장되고, 최종적으로 앱 알림으로 전송되는 전체 연동 과정을 확인하였다.

아울러 야간과 악천후 환경, 원거리 촬영, 빠른 속도로 이동하는 동물 등 다양한 조건에서 환경 기반 실험을 수행하였으며 오탐과 미탐 비율 분석과 알림 지연 시간 측정을 통해 실사용 환경에서의 성능을 평가하였다. 이러한 검증 과정을 통해 SENCITY는 AI 모델 개발부터 스트리밍 영상 및 이미지 수집, 서버 처리, 모바일 알림에 이르기까지 AI-IoT-서버-앱이 유기적으로 통합된 실제 운영 가능한 수준의 시스템임을 확인하였으며, 정량적 성능 지표와 실환경 테스트 모두에서 높은 신뢰도와 완성도를 확보하였다.

7. 기대효과 및 사회적 가치

1. 기대효과

본 프로젝트를 통해 도심 내 야생동물 출현 정보를 실시간으로 수집하고 분석하며 전달하는 체계가 구축되어, 시민의 수동 신고에 의존하던 기존 대응 방식에서 발생하던 지연과 누락 문제를 크게 줄일 수 있다. AI 기반 자동 탐지와 신속한 알림 시스템을 통해 시민과 지자체가 동시에 상황을 인지하고 즉각적으로 대응할 수 있으며, 이는 단순한 정보 전달을 넘어 지역 사회가 하나의 안전 네트워크로 작동하는 기반을 마련한다. 이를 통해 도심 내 사고 예방과 시민 안전 확보에 실질적으로 기여할 수 있다.

또한 자동으로 생성되고 축적되는 출몰 데이터는 지역별 발생 빈도, 시간대별 패턴, 종별 분포를 정량적으로 분석할 수 있게 하며, 데이터가 장기적으로 누적될수록 계절성 변화나 위험도 추이, 종 이동 경향과 같은 고도화된 분석도 가능해진다. 이러한 분석 결과는 도시 생태계 관리와 위험 지역 예측, 예방 중심의 정책 수립 등 공공 의사결정 과정에 활용될 수 있으며, 과학적 근거에 기반한 지속 가능한 환경 관리로 이어진다. 결과적으로 SENCITY는 실시간 대응 체계 구축과 데이터 자산 축적을 동시에 실현하는 플랫폼으로서, 도심 야생동물 문제 해결에 있어 장기적이고 구조적인 기대효과를 가진다.

1) 시민 안전 확보

SENCITY는 실시간 감지와 즉각적인 경보 체계를 통해 도심 내 야생동물 출몰로 발생할 수 있는 사고 위험을 크게 줄인다.

- 사전 경보 공: 위험 지역 접근 전 푸시 알림이 전달되어 교통사고·접촉사고 예방 효과가 크다
- 취약계층 보호 강화: 등하굣길 어린이, 노약자, 야간 이동이 많은 시민에게 특히 높은 보호 효과를 제공한다.
- 일상 속 안심 제공: 야간 산책·조깅·반려견 산책 등에서의 심리적 불안을 감소시킨다.

2) 효율적인 야생동물 관리

AI 기반 자동 감지 시스템을 도입함으로써 기존 수동 점검 방식 대비 관리 효율이 증가한다.

- 신속한 현장 대응: 관리자 대시보드를 통해 출몰 위치, 시간, 종 정보가 즉시 제공되어 현장 출동 의사결정이 빨라진다.
- 장기적 정책 수립 가능: 지속적으로 수집된 데이터(출몰 패턴·시간대·계절별 차이)를 분석하여 지자체의 야생동물 관리 계획 수립에 활용할 수 있다.
- 관리 비용 절감: 기존에는 다수의 인력이 순찰, 확인, CCTV 모니터링을 담당해야 했지만 자동화 시스템 도입으로 인력 소모를 크게 줄일 수 있다.
- (예: 출몰 의심 민원 확인 인력 최소 30~50% 감소 가능)

3) 데이터 기반 정책 지원

체계적으로 수집 및 보관된 출몰 데이터는 지자체 정책의 근거 자료로 활용된다.

- 정량적 출몰 분석: 시간대·지역별 패턴을 근거로 집중 관리 대상 구역을 지정할 수 있다.
- 이동 경로 분석: 반복 출몰 지점을 통해 이동 경로와 서식지 변화를 파악할 수 있다.
- 도시 계획 반영: 도로 신설, 거주지 확장, 공원 조성 등 개발 과정에 생태 데이터를 적용할 수 있다.

4) 환경 보호 및 공존 문화 형성

SENCITY는 단순한 감지/경보 시스템을 넘어 인간과 야생동물의 안전한 공존을 도모하는 플랫폼이다.

- 시민 인식 개선: 앱을 통해 종별 정보·대처법을 제공하여 생태 교육 효과를 높인다.
- 과학적 관리 체계 구축: 감각적 대응이 아닌 정량 데이터 기반의 의사결정 지원
- 생태계 보호 기여: 장기 모니터링으로 서식지 교란을 줄이고 도시 생태 균형을 유지한다.

2. 사회적 가치

SENCITY는 Vision AI와 IoT 기술을 활용하여 도심 내 위험 상황을 자동으로 인식하고 공유함으로써, 사용자 신고에 의존하던 기존 시스템의 한계를 극복하고 예방 중심의 대응 체계를 구축한다. 이를 통해 지역 사회의 대응 속도와 신뢰도를 높이고, 사고 발생 이전 단계에서 위험을 인지하고 대응할 수 있는 환경을 조성한다. 자동으로 축적되는 출몰 기록과 분석된 통계 데이터는 지자체가 과학적 근거에 기반한 예방 정책을 수립하도록 지원하며, 시민 안전 강화와 서식지 보전 등 다양한 사회적 가치를 창출한다.

또한 SENCITY는 축적된 데이터를 활용해 지역 간 출몰 패턴 비교, 고위험 지역의 조기 탐지, 계절성 대응 계획 수립이 가능하도록 하여 야생동물 관리의 효율성을 높인다. 이러한 구조는 특정 지역에 국한되지 않고 다른 지자체로 확산 가능한 데이터 기반 공공 서비스 모델로서의 가능성을 가지며, 장기적으로는 전국 단위의 안전 관리 체계와 도시 생태 및 환경 안전 관리 역량을 강화하는 데 기여할 것으로 기대된다.

1) 공공 안전 향상

서울시 기준 연간 800건 이상 발생하는 출몰 관련 사도를 20~40% 감소시킬 잠재력이 있다. 사고 1건당 약 50만원이 소요되는 출동 비용을 크게 절감하여, 연간 수억 원의 예산 절약이 가능하다.

2) 스마트시티 생태계 기여

AI, IoT, GIS, 빅데이터 기반 통합 플랫폼으로, 지자체가 즉시 활용 가능한 스마트시티 레퍼런스가 된다.

- 자동 감지 + 실시간 지도 기반 모니터링
- 타 지자체 벤치마킹 및 연동 용이
- 공공 데이터 혁신 사례로 활용 가능

3) 연구 및 교육 자원

장기 축적된 데이터는 학문적 가치가 높으며 연구·교육 목적으로 활용할 수 있다.

- 생태학 및 동물 행동학 연구의 기초 데이터
- 학교/환경단체 교육 프로그램 자료
- 지역 생태 변화 추적 및 정책 연구 활용

8. 외부 활동 및 성과

1. 공모전 수상

- 한국정보통신보안윤리학회 “지역혁신 인재양성” 연합 페스티벌 공모전
 - 지역사회문제해결 아이디어 부문 우수상
 - AI/DX 산출물 부문 우수상

지역사회문제해결 아이디어 부문	AI/DX 산출물 부문
제 2025-096호 2025년도 “지역혁신 인재양성” 연합 페스티벌 상 장 지역사회문제해결 부문 우수상 소속 : 한양여자대학교 이름 : 신지윤, 박용아 위 사람은 한국정보통신보안윤리학회와 전국의 대학교가 함께 개최한 “2025년도 지역혁신 인재양성 연합 페스티벌 공모전” 에서 창의적이고 탁월한 아이디어를 제시하였기에 이 상장을 수여합니다. 2025년 11월 14일 사단법인 한국정보통신보안윤리학회 회장 유진호	제 2025-104호 2025년도 “지역혁신 인재양성” 연합 페스티벌 상 장 AI·DX 산출물 부문 우수상 소속 : 한양여자대학교 이름 : 신지윤, 박용아 위 사람은 한국정보통신보안윤리학회와 전국의 대학교가 함께 개최한 “2025년도 지역혁신 인재양성 연합 페스티벌 공모전” 에서 창의적이고 탁월한 아이디어를 제시하였기에 이 상장을 수여합니다. 2025년 11월 14일 사단법인 한국정보통신보안윤리학회 회장 유진호

2. 프로젝트 결과물

- [프로젝트 PPT](#)
- [시연 영상](#)
- [GitHub](#)
- 프로젝트 전시 포스터

HANYANG WOMEN'S UNIVERSITY / DEPARTMENT OF BIGDATA
SHIN JIYOON / PARK YOON AH / YUN NU RI

SENCITY

도심 내 야생동물 출몰 알림 및 신고 앱

INTRODUCTION

● 주제 선정 배경

급격한 기후 변화와 도시 확장에 따른 서식지 감소로 도심 내 야생동물의 출현 빈도가 지속적으로 증가하고 있다. 서울시 25개 자치구 중 24개 구에서 나무가 관할되었으며, 이러한 출몰 증가 현상은 인명 피해, 로드킬, 반려동물 감염 같은 다양한 사회적 위험으로 이어지고 있다. 이에 따라 지자체와 민간 기업은 중심으로 여러 관리 시도가 이루어지고 있으나, 현재 제공되는 서비스들은 출몰 정보를 체계적으로 수집·분석하기에는 여전히 구조적 한계를 지닌다.

OBJECTIVES

본 프로젝트는 AI 기반 자동 탐지/분석/알림 시스템 'SENCITY'를 통해 도심 내 야생동물 출현 문제를 능동적으로 대응하고자 한다. 플랫폼이 촬영한 영상 데이터는 Django 서버로 전송되며, YOLOv5s 객체 탐지 모델이 이를 실시간으로 분석해 야생동물의 종류와 위치를 판별한다. 탐지된 결과는 Flutter 앱으로 즉시 전달되어 앱에서 자동 탐지 기록을 확인할 수 있다. 또한 직접 사진을 업로드해 데이터를 제공할 수 있으며, 업로드된 이미지는 EfficientNet-B1 분류 모델이 처리하여 종을 식별한다. 이 과정은 데이터 축적·분석·모델 고도화로 이어지는 순환 구조를 형성한다. SENCITY의 자동 탐지/분석/전송 체계는 기존의 수동 신고 방식에 의존하지 않고 출몰 정보를 지속적으로 확보할 수 있어 사고 예방에 효과적이며, 장기적으로는 도시 안전 강화와 생태계 보전을 동시에 달성하는 기반을 마련한다.

METHODS

RESULTS 01

RESULTS 02

CONCLUSIONS

본 프로젝트는 도심 내 야생동물 출현에 대한 실시간 대응 체계 구축을 가능하게 한다. 기존처럼 사용자의 수동 신고에 의존하던 방식에서 벗어나 자동 탐지 및 알림 시스템을 구현함으로써 대응 공백을 최소화할 수 있으며, 탐지 결과가 시민과 지자체 모두에게 즉시 전달되어 신속한 상황 인지가 가능해진다. 또한 관리자 출현 데이터를 체계적으로 저장하고 분석함으로써 지역별 출현 빈도, 시간대별 패턴, 종별 분포 등 다양한 지표를 정량적으로 확보할 수 있다. 이 데이터는 향후 도시 생태계 관리, 출몰 위험 지역 지정, 안내 표지 설치, 야생동물 관리 정책 수립 등 실질적인 의사결정에 활용될 수 있으며, 장기적으로는 과학적 근거 기반의 지속 가능한 도시와 환경 관리 체계 구축에 기여한다.

CONCLUSIONS

본 프로젝트는 도심 내 야생동물 출현에 대한 실시간 대응 체계 구축을 가능하게 한다. 기존처럼 사용자의 수동 신고에 의존하던 방식에서 벗어나 자동 탐지 및 알림 시스템을 구현함으로써 대응 공백을 최소화할 수 있으며, 탐지 결과가 시민과 지자체 모두에게 즉시 전달되어 신속한 상황 인지가 가능해진다. 또한 관리자 출현 데이터를 체계적으로 저장하고 분석함으로써 지역별 출현 빈도, 시간대별 패턴, 종별 분포 등 다양한 지표를 정량적으로 확보할 수 있다. 이 데이터는 향후 도시 생태계 관리, 출몰 위험 지역 지정, 안내 표지 설치, 야생동물 관리 정책 수립 등 실질적인 의사결정에 활용될 수 있으며, 장기적으로는 과학적 근거 기반의 지속 가능한 도시와 환경 관리 체계 구축에 기여한다.

DISCUSSION

현재는 제한된 지역 환경과 일부 종을 대상으로 실증이 이루어졌기 때문에, 다양한 지형/기후/계절 변화에서도 안정적인 성능을 유지하는지에 대한 추가 검증이 필요하다. 특히 산간 지역이나 동식물 인프라이기 위한 환경에서는 데이터 전송 지연과 손실이 발생될 수 있으며, 기상 조건이나 조도 변화에 따라 영상 품질이 달라지면서 AI 모델의 인식률이 저하되는 문제도 확인되었다.

이를 보완하기 위해 탐지 모델을 YOLOv5s로 전환하여 정확도와 인식 안정성을 높이고 야간, 악천후 등 다양한 환경 데이터를 포함한 추가 학습을 수행할 예정이다. 더불어 다양한 카메라와 센서 장치에서도 일관된 인식 품질을 유지할 수 있도록 시스템 전반의 최적화를 지속적으로 추진함으로써 실제 운영 환경에서도 높은 신뢰도를 확보하고자 한다.

References

1. kaggle, 'AwA2_dataset', <https://www.kaggle.com/datasets/radmik2/awa2-dataset>

2. 이원진, 박수민, 홍정수, "YOLO를 이용한 야생동물 탐지 및 위치 추적 시스템 개발", 한국정보과학회 2018 한국소프트웨어융합학술대회 논문집, p1,897-1,899

9. 한계점 및 향후 개선 방향

1. 한계점

1) 야간 및 악천후 정확도 저하

SENCITY의 AI 모델은 주간 환경에서는 높은 정확도를 유지하지만, 조도 부족 및 기상 악화 시 탐지 성능이 최대 78.3%까지 하락하는 한계를 보인다.

- 야간 촬영 시 노이즈 증가, 모션 블러 등으로 객체 경계가 흐려짐
- 비/눈/안개와 같은 기상 조건에서는 카메라 센서 자체의 영상 품질이 급격히 낮아짐

이러한 요인들은 YOLO 기반 모델의 bounding box 생성 및 confidence score에 직접적인 영향을 미친다

2) 오탐 발생

AI 모델이 특정 조건에서는 비대상 객체를 야생동물로 오감지하거나 유사 체형의 동물을 서로 혼동하는 사례가 존재한다.

- 너구리 → 고양이, 고라니 → 강아지 등 색상/귀 모양 등이 비슷한 종 간 오분류
- 사람/반려동물/자전거 등이 약 8.3% 비율로 오탐지

이는 실제 현장 대응을 혼란시키고 신고 시스템의 신뢰도에도 영향을 줄 수 있다.

3) 서버 확장성

현재 시스템은 소규모 지역 기준으로 안정적인 운영이 가능하나, 다수의 CCTV 영상이 동시에 입력될 경우 서버 부하가 증가할 수 있다.

- 실시간 영상 8대 이상 처리 시 GPU/CPU 자원 사용량 급증
- 지역 전체에 적용할 경우 클라우드 기반 오토스케일링 환경이 필요

서버 부하로 인해 분석 지연이 발생할 경우 실시간성이 떨어질 위험이 있다.

2. 향후 개선 방향

향후 SENCITY는 자동 신고 기능을 더욱 고도화하여, 관리자 승인 없이도 YOLOv5s 모델이 신고가 필요한 야생동물을 탐지하는 즉시 도로교통공사나 지역 구조센터 등 관련 기관으로 자동 신고가 전송되는 체계를 구축하고자 한다. 이를 통해 기존 수동 신고 과정에서 발생하던 지연과 누락을 최소화하고, 실제 현장에서 즉각적인 대응이 이루어질 수 있는 자동 대응 환경을 강화할 계획이다.

또한 EfficientNet-B1과 YOLOv5s 모델의 전반적인 분류 및 탐지 정확도를 지속적으로 향상시키고, 특히 야간이나 악천후와 같이 영상 품질이 저하되는 환경에서도 안정적인 인식이 가능하도록 모델의 환경 적응력을 강화한다. 이를 위해 적외선 카메라와 열화상 카메라 도입을 검토하고, 야간 및 악천후 이미지 데이터를 대규모로 추가 확보하여 재학습을 진행하며, 밝기 보정, 감마 보정, CLAHE와 같은 영상 개선 알고리즘을 적용해 AI 분석에 최적화된 입력 영상을 제공할 예정이다. 더불어 NMS 임계값 튜닝, YOLO와 이미지 분류 모델을 결합한 앙상블 구조, 프레임 간 정보를 활용한 시계열 필터링 기법을 적용하여 오탐을 줄이고 인식 안정성을 높이하고자 한다.

사용자 참여 기반의 시스템 또한 강화된다. 커뮤니티 기능과 포인트, 레벨, 리워드와 같은 보상 체계를 도입해 사용자의 제보와 지역 정보 공유를 유도함으로써, 서비스 운영에 필요한 데이터가 지속적으로 축적되는 구조를 구축한다. 이는 데이터 양과 다양성을 동시에 확보하여 AI 성능 개선과 서비스 활성화로 이어지는 선순환 구조를 형성하는 데 기여할 것이다.

아울러 현재 10종 수준의 야생동물 분류 범위를 20종 이상으로 확장하고, 조류를 포함한 다양한 종을 대상으로 관리 범위를 넓혀 개체수 변화와 확산 속도 추적까지 가능하도록 할 계획이다. 축적된 출몰 데이터를 기반으로 시계열 예측 모델을 적용해 특정 시간대와 지역의 출몰 확률을 사전에 예측하는 기능을 개발함으로써, 지자체가 순찰 강화나 안내 조치와 같은 예방적 대응을 사전에 수행할 수 있도록 지원하고자 한다.

장기적으로는 SENCITY를 단순한 안전 서비스에 그치지 않고, 축적된 데이터를 활용한 생태계 연구 플랫폼으로 확장하는 것을 목표로 한다. 이를 통해 연구기관이나 대학과의 협업을 바탕으로 기후 변화와 야생동물 이동 패턴 간의 상관관계를 분석하고, 도시 생태계 변화 추적을 위한 데이터베이스로 활용함으로써 공공 안전과 학술적 가치가 함께 증대되는 방향으로 발전시키고자 한다.

10. 결론

1. 결론 및 고찰

본 프로젝트 SENCITY는 EfficientNet-B1 이미지 분류 모델과 YOLOv5s 객체 탐지 모델을 결합하여 도심 내 야생동물 출몰 문제에 대한 선제적 대응 체계를 구현한 사례로서, AI 기술을 실제 도시 안전 문제 해결에 적용했다는 점에서 의미를 가진다. 10종 야생동물에 대한 자동 식별과 실시간 영상 탐지, 위치 기반 알림 시스템을 하나의 파이프라인으로 통합함으로써 감지 결과가 사용자와 관리자에게 동시에 전달되는 통합 안전 관리 플랫폼을 구현하였으며, 이를 통해 시민 안전 확보와 지자체의 신속한 현장 대응, 장기적인 생태 및 정책 데이터 축적이 가능해졌다. 특히 본 시스템은 단순한 시뮬레이션 수준을 넘어 실제 환경에서 즉시 활용 가능한 구조로 설계되었다는 점에서 실용적 의의를 지닌다.

개발 과정에서는 AI 모델 학습과 데이터 수집 및 라벨링, 백엔드와 프론트엔드 연동, IoT 기반 영상 입력 처리 등 다양한 기술 요소를 직접 통합하며 이론 중심으로 학습했던 AI와 데이터베이스 개념을 실무 관점에서 이해할 수 있었다. Django 서버와 Flutter 기반 모바일 앱, AI 분석 모듈을 유기적으로 연결하는 과정을 통해 AI 모델이 실제 서비스 환경에서 어떻게 동작하고, 데이터가 어떤 흐름으로 수집되고 활용되는지를 체감할 수 있었다. 이 과정에서 단순한 기능 구현을 넘어 문제를 구조적으로 정의하고 요구 사항에 맞는 기술을 선택하여 하나의 시스템으로 설계하는 과정의 중요성을 인식하게 되었으며, 이는 AI 기반 서비스 개발에 필요한 종합적인 실무 역량을 향상시키는 계기가 되었다.

한편 본 프로젝트는 몇 가지 한계점도 함께 확인할 수 있었다. 야간이나 악천후와 같이 영상 품질이 저하되는 환경에서는 인식 정확도가 낮아질 가능성이 있으며, 대규모 지역에 적용할 경우 서버 부하와 같은 확장성 문제도 발생할 수 있다. 또한 현재는 제한된 종을 대상으로 실험이 이루어졌기 때문에 다양한 종과 환경 조건을 포괄하기 위해서는 추가적인 데이터 확보와 모델 고도화가 필요하다.

이에 따라 향후에는 야간 및 악천후 환경에서의 인식 정확도 개선을 위한 데이터 확장과 모델 보완, 클라우드 기반 인프라 도입을 통한 확장성 확보, 동물 종 확대와 출몰 예측 모델 적용을 통해 서비스 활용 범위를 넓힐 계획이다. 더 나아가 축적된 출몰 데이터를 기반으로 예방 중심의 대응 체계를 강화함으로써, 궁극적으로는 사람과 야생동물이 안전하게 공존할 수 있는 도시 환경 조성에 기여하는 것을 목표로 한다.

11. 참고 문헌

- 보도 자료

1. [도심 너구리에 피부병,관경병..."관리 위해 생태정보 필요" 미디어펜 이소희 기자](#)
2. [도심 공원에 '벌러덩' 너구리...기생충,광견병 우려 채널 A 김민환 기자](#)
3. [서울시, 도심 출몰 너구리 대상 인수공통감염병 첫 정밀 검사 착수 서울특별시 보건환경연구원 보도자료](#)
4. [도심의 습격자, 전염병 숙주? 너구리는 억울하다 \[위기의 도심동물들\] 한국일보 고은경 기자](#)
5. [사람이 주는 먹이에 도심으로 모이는 너구리들 연합뉴스](#)
6. [\[정책 인사이트\] 서울 누비는 야생동물... 암사동에 삶, 연대 뒷산에 산양, 남산에 솔부엉이 조선일보 손덕호, 홍다영 기자](#)
7. [서울에 출몰하는 거대 야생동물...누구나, 넌! 내 손안에 서울 뉴스를 광재식 교수](#)

- 데이터셋

1. [Kaggle 너구리, 다람쥐, 족제비 이미지 데이터](#)
2. [Kaggle 멧토끼, 너구리, 다람쥐, 족제비 이미지 데이터](#)
3. [Kaggle 반달가슴곰 이미지 데이터](#)
4. [AIHub 야생동물 활동 영상 데이터](#)

- 논문

[이아현, 박수민, 홍정수. 「YOLO를 이용한 야생동물 접근 및 피해 예방 시스템 개발」. 한국정보과학회 2018 한국소프트웨어종합학술대회 논문집. p1,897 - 1,899](#)